

Rapport de Stage de Fin d'Études

Réalisé au sein de : FRS



Titre du projet :

**Conception et Développement d'un Système CRM Full-Stack
avec Portail Client, Application Mobile et Chatbot IA**

Préparé par

Mahdi CHAKROUN

**Encadrant académique :
Ahmed KHARAT**

**Encadrant professionnel :
Ghassen BAKLOUTI**

Année Universitaire : 2025 / 2026



DÉDICACE

Avec l'expression de ma reconnaissance, je dédie ce modeste travail à ceux qui, quels que soient les termes embrassés, je n'arriverai jamais à leur exprimer mon amour sincère.

A l'homme, mon précieux don de Dieu, à qui je dois ma vie,
ma réussite et tout mon respect : **mon cher père.**

A la femme qui a souffert sans me laisser souffrir, qui n'a jamais dit non à mes exigences et qui n'a épargné aucun effort pour me rendre heureux : **mon adorable mère.**

A mes adorables frères et sœurs, qui savent toujours comment procurer la joie et le bonheur à toute la famille.

A tous les amis que j'ai connus jusqu'à maintenant. Merci pour leur amour et leurs encouragements.

A vous tous,

Je dédie ce travail.



REMERCIEMENTS

Je tiens à exprimer mes sincères remerciements à mon encadrant académique, Monsieur **Ahmed KHARAT**, ainsi qu'à mon encadrant professionnel, Monsieur **Ghassen BAKLOUTI**, pour leur accompagnement tout au long de ce stage, leur disponibilité, ainsi que leurs conseils précieux. Grâce à leur encadrement, j'ai pu mieux comprendre les enjeux métiers liés à la gestion de la relation client et progresser méthodiquement dans les différentes étapes du projet.

J'adresse également mes remerciements à l'ensemble de l'équipe de **FRS** pour leur accueil chaleureux, leur soutien et le cadre de travail professionnel mis à ma disposition.

Je remercie par ailleurs mes enseignants de **l'Institut Polytechnique Privé des Sciences Avancées de Sfax (IPSAS)** pour la qualité de la formation dispensée, qui m'a permis de mobiliser les compétences nécessaires à la réussite de ce stage.

Enfin, j'exprime ma gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail.



RÉSUMÉ

Ce rapport présente le travail réalisé dans le cadre d'un stage de fin d'études effectué au sein de l'entreprise **FRS**. Le projet consiste à concevoir et développer un système CRM complet destiné à centraliser les activités commerciales et le support client au sein d'une même plateforme. L'objectif principal est de proposer une solution cohérente, moderne et adaptable, couvrant la gestion des prospects, des comptes, des contacts, des opportunités commerciales, des devis, des tickets de support et du suivi des activités.

La solution livrée s'articule autour de plusieurs composantes complémentaires. Une application web constitue le coeur du CRM interne pour les rôles administratifs, commerciaux, support et managériaux. Un portail client sécurisé permet aux clients de consulter leurs devis et leurs tickets. Une application mobile, développée avec React Native et Expo, étend ce portail sur smartphone afin d'offrir un accès nomade aux fonctionnalités essentielles côté client. En complément, un assistant conversationnel basé sur la technique **RAG** facilite l'accès à l'information métier à partir d'une base documentaire dédiée regroupant la présentation de l'entreprise, les services, la FAQ, les conditions générales, le guide support et des références projets.

Sur le plan technique, le système repose sur un backend **FastAPI** structuré par domaines métier, une base de données relationnelle **MySQL**, un frontend **React** construit avec **Vite**, ainsi qu'une application mobile **React Native**. L'authentification est assurée par JWT et la séparation des responsabilités s'appuie sur un mécanisme RBAC à cinq rôles. Le module d'assistance utilise **ChromaDB** comme base vectorielle et peut s'appuyer sur un LLM externe lorsque celui-ci est configuré. L'ensemble du projet est orchestré via **Docker Compose**, ce qui facilite les phases de démonstration, de test et de déploiement local.

Mots-clés : CRM, FastAPI, React, React Native, MySQL, Docker Compose, RAG, ChromaDB, Portail Client, Support Client.

ABSTRACT

This report presents the work carried out during a final-year internship at **FRS**. The project focuses on the design and implementation of a complete CRM platform intended to centralize sales management and customer support within a single information system. The main objective

is to provide a consistent and extensible solution covering lead management, customer accounts, contacts, business opportunities, quotations, support tickets and activity tracking.

The delivered solution is composed of several complementary parts. A web application serves as the internal CRM for administrative, managerial, sales and support teams. A secured customer portal allows clients to access their quotations and support tickets. A mobile application built with React Native and Expo extends this portal on smartphones in order to provide mobile access to essential client-side features. In addition, a conversational assistant based on the **RAG** approach helps users retrieve business information from a dedicated document base including company presentation, service catalog, FAQ, terms and conditions, support guide and project references.

From a technical perspective, the system relies on a **FastAPI** backend organized by business domains, a **MySQL** relational database, a **React** frontend built with **Vite**, and a **React Native** mobile application. Authentication is handled through JWT and access control is enforced with a five-role RBAC model. The assistant uses **ChromaDB** as a vector store and can leverage an external LLM when configured. The whole project is orchestrated with **Docker Compose**, which simplifies local deployment, testing and demonstration.

Keywords : CRM, FastAPI, React, React Native, MySQL, Docker Compose, RAG, ChromaDB, Customer Portal, Customer Support.



TABLE DES MATIÈRES

LISTE DES FIGURES	viii
LISTE DES TABLEAUX	ix
LISTE DES ABRÉVIATIONS	x
INTRODUCTION GÉNÉRALE	xii
1 Cadre Général du Projet	1
1.1 Introduction	2
1.2 Contexte du projet	2
1.3 Présentation de l'organisme d'accueil	3
1.4 Problématique	3
1.5 Approche proposée	4
1.5.1 Fonctionnalités du CRM Web	4
1.5.2 Fonctionnalités de l'Application Mobile	5
1.5.3 Fonctionnalités du Chatbot IA	5
1.6 Méthodologie	5
1.6.1 Méthodologie Agile Scrum	5
1.6.2 Méthodologie CRISP-DM	6
1.6.3 Organisation en sprints	6
1.7 Architecture globale	7
1.8 Environnement technique	8
1.9 Conclusion	10
2 Sprint 01 : Étude Préalable et Analyse	12
2.1 Introduction	13
2.2 Objectifs du Sprint	13
2.3 Étude de l'existant	13
2.3.1 Analyse des solutions CRM existantes	13
2.3.2 Critique de l'existant	14
2.4 Identification des acteurs	15
2.5 Analyse des besoins fonctionnels	15
2.5.1 Module Authentification et Sécurité	15
2.5.2 Module Leads (Prospects)	16
2.5.3 Module Accounts (Comptes Clients)	16
2.5.4 Module Contacts (Personnes)	16
2.5.5 Module Deals (Pipeline Commercial)	16
2.5.6 Module Quotes (Devis)	17
2.5.7 Module Tickets (Helpdesk)	17

2.5.8	Module Activities (Suivi Commercial)	17
2.5.9	Module Reporting et Audit	17
2.5.10	Module Portail Client	17
2.5.11	Module Application Mobile	18
2.5.12	Module Chatbot IA (RAG)	18
2.6	Besoins non fonctionnels	18
2.7	Modélisation UML — Diagrammes de cas d'utilisation	19
2.7.1	Diagramme de cas d'utilisation global	19
2.7.2	Cas d'utilisation — Gestion des Leads	21
2.7.3	Cas d'utilisation — Gestion des Tickets	21
2.7.4	Cas d'utilisation — Portail Client	22
2.7.5	Cas d'utilisation — Application Mobile	24
2.7.6	Cas d'utilisation — Chatbot IA	25
2.8	Backlog produit	25
2.9	Résultats du Sprint	26
2.10	Conclusion	26
3	Sprint 02 : Conception du Système CRM	28
3.1	Introduction	29
3.2	Objectifs du Sprint	29
3.3	Diagramme de classes	29
3.3.1	Relations entre entités	30
3.4	Diagramme Entité-Relation (ERD)	31
3.5	Diagrammes de séquence	31
3.5.1	Séquence — Authentification JWT	31
3.5.2	Séquence — Conversion d'un Lead	32
3.5.3	Séquence — Cycle de vie d'un Ticket	32
3.5.4	Séquence — Chatbot documentaire (RAG)	33
3.6	Conception de l'architecture	33
3.6.1	Architecture backend — FastAPI + organisation par domaines	33
3.6.2	Architecture frontend — React SPA	34
3.6.3	Architecture mobile — React Native	34
3.6.4	Architecture RAG — Chatbot IA	35
3.6.5	Architecture Docker Compose	35
3.7	Conception de l'API REST	36
3.8	Résultats du Sprint	37
3.9	Conclusion	37
4	Sprint 03 : Réalisation Technique	39
4.1	Introduction	40
4.2	Objectifs du Sprint	40
4.3	Réalisation du Backend (FastAPI)	40
4.3.1	Authentification JWT et RBAC	40
4.3.2	Logique de conversion Lead	41
4.3.3	Système de messagerie Ticket	41
4.3.4	Journal d'audit et reporting	41
4.3.5	Notifications email	42
4.4	Réalisation du Frontend (React)	42

4.4.1	Pages principales	42
4.4.2	Portail Client	44
4.4.3	Fonctionnalités UX avancées	44
4.5	Réalisation de l'Application Mobile (React Native)	45
4.5.1	Technologies utilisées	45
4.5.2	Écrans de l'application	45
4.6	Réalisation du Chatbot IA (RAG)	46
4.6.1	Architecture RAG	46
4.6.2	Implémentation	46
4.6.3	Documents RAG	47
4.7	Déploiement Docker	47
4.8	Résultats du Sprint	48
4.9	Conclusion	48
5	Sprint 04 : Tests, Validation et Déploiement	50
5.1	Introduction	51
5.2	Objectifs du Sprint	51
5.3	Tests fonctionnels	51
5.3.1	Dispositif de tests automatisés	51
5.3.2	Matrice de tests RBAC	52
5.3.3	Scénario de test end-to-end	52
5.4	Bugs identifiés et corrigés	52
5.4.1	Méthodologie de correction	53
5.5	Validation du chatbot IA	53
5.6	Résultats du Sprint	54
5.7	Conclusion	54
	CONCLUSION GÉNÉRALE	56
	BIBLIOGRAPHIE	58

LISTE DES FIGURES

1.1	Impact d'une gestion dispersée de la relation client	2
1.2	Logo de l'entreprise FRS	3
1.3	Cycle général de la méthodologie Scrum	6
1.4	Cycle CRISP-DM mobilisé pour la préparation de la base documentaire	6
1.5	Architecture globale du système CRM	7
2.1	Diagramme de cas d'utilisation global du système CRM	20
2.2	Cas d'utilisation du module Leads	21
2.3	Cas d'utilisation du module Tickets	22
2.4	Cas d'utilisation du portail client	23
2.5	Cas d'utilisation de l'application mobile	24
2.6	Cas d'utilisation du chatbot documentaire (RAG)	25
3.1	Diagramme de classes du système CRM	30
3.2	Diagramme Entité-Relation de la base CRM	31
3.3	Diagramme de séquence de l'authentification JWT	32
3.4	Diagramme de séquence de conversion d'un lead	32
3.5	Diagramme de séquence du cycle de vie d'un ticket	33
3.6	Diagramme de séquence du chatbot documentaire	33
3.7	Architecture du module RAG	35
4.1	Capture d'écran — Page de connexion	43
4.2	Capture d'écran — Tableau de bord interne	43
4.3	Capture d'écran — Module Leads / Deals	43
4.4	Capture d'écran — Module Tickets	44
4.5	Capture d'écran — Portail client web	44
4.6	Captures d'écran — Application mobile	46
4.7	Capture d'écran — Assistant documentaire	47

LISTE DES TABLEAUX

1.1	Fiche signalétique de l'organisme d'accueil	3
1.2	Composantes fonctionnelles de la solution proposée	4
1.3	Organisation du projet en sprints	7
1.4	Planification simplifiée du projet	8
1.5	Environnement technique du projet	9
2.1	Comparaison synthétique entre solutions existantes et solution développée . . .	14
2.2	Acteurs du système et responsabilités associées	15
2.3	Principaux besoins non fonctionnels	19
2.4	Backlog produit synthétique	26
2.5	Livrables du Sprint 01	26
3.1	Entités principales du diagramme de classes	29
3.2	Principales relations métier du système	30
3.3	Organisation logique du backend	34
3.4	Principes de conception de l'application mobile	35
3.5	Services définis dans l'architecture Docker Compose	36
3.6	Extrait des principaux endpoints de l'API REST	36
3.7	Livrables du Sprint 02	37
4.1	Principes de réalisation du module d'authentification	41
4.2	Organisation fonctionnelle du frontend web	42
4.3	Écrans principaux de l'application mobile	45
4.4	Composants techniques du module RAG	46
4.5	Documents utilisés par la base de connaissances documentaire	47
4.6	Services de l'infrastructure Docker Compose	48
4.7	Livrables du Sprint 03	48
5.1	Stratégie de tests du projet	51
5.2	Synthèse des permissions RBAC	52
5.3	Exemples d'anomalies de stabilisation traitées	53
5.4	Validation fonctionnelle du module d'assistance documentaire	54
5.5	Livrables du Sprint 04	54



LISTE DES ABRÉVIATIONS

API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CRM	Customer Relationship Management
CSS	Cascading Style Sheets
DDD	Domain-Driven Design
DTO	Data Transfer Object
ERD	Entity-Relationship Diagram
IA	Intelligence Artificielle
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model
ORM	Object-Relational Mapping
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
SLA	Service Level Agreement
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
UX	User Experience

Introduction Générale



Introduction Générale

Dans un environnement économique marqué par une forte concurrence et par une exigence croissante en matière de qualité de service, la gestion de la relation client s'impose comme un enjeu stratégique majeur. Les entreprises ne peuvent plus se contenter d'outils dispersés pour suivre leurs prospects, leurs opportunités commerciales, leurs interactions de support et leurs engagements contractuels. Elles ont besoin d'un système centralisé, capable de structurer l'information, d'améliorer la coordination entre équipes et de fournir une vision fiable de l'activité en temps réel.

Les solutions CRM du marché répondent en partie à cette problématique, mais elles restent souvent coûteuses, surdimensionnées ou difficiles à adapter aux contraintes spécifiques d'une entreprise donnée. Dans ce contexte, le projet réalisé au sein de **FRS** vise à concevoir une solution maîtrisée, personnalisable et cohérente avec les besoins réels de l'organisation. L'ambition n'est pas seulement de reproduire des fonctionnalités classiques de prospection ou de ticketing, mais de bâtir un écosystème numérique intégrant à la fois le travail interne, le suivi client et l'accès à l'information métier.

Le travail mené durant ce stage aboutit ainsi à la réalisation d'un système CRM articulé autour de quatre composantes complémentaires :

- un **CRM web interne** pour les équipes administratives, commerciales, support et managériales ;
- un **portail client web** dédié à la consultation des tickets, devis et informations personnelles ;
- une **application mobile** orientée portail client pour l'accès nomade aux fonctionnalités essentielles ;
- un **assistant conversationnel** fondé sur une base documentaire et la technique RAG.

L'application web couvre les principaux besoins métier d'un CRM moderne : authentification et gestion des rôles, suivi des leads, création des comptes et contacts, pilotage du pipeline commercial, gestion des devis, traitement des tickets, suivi des activités, journalisation des actions et génération de tableaux de bord. Le portail client et l'application mobile prolongent cette logique côté utilisateur final, tandis que le chatbot apporte une couche d'assistance documentaire pour fluidifier l'accès à l'information.

La réalisation du projet s'inscrit dans une démarche **Agile Scrum** structurée en quatre sprints successifs :

- **Sprint 01** : étude préalable, analyse des besoins, identification des acteurs et élaboration du backlog ;
- **Sprint 02** : conception détaillée, modélisation UML, architecture technique et conception de l'API ;
- **Sprint 03** : réalisation technique du backend, du frontend, de l'application mobile et du module d'assistance ;
- **Sprint 04** : tests, validation, corrections de stabilisation et préparation du déploiement local conteneurisé.

Le présent rapport est organisé en cinq chapitres. Le premier chapitre présente le cadre général du projet et son environnement. Le deuxième chapitre expose l'étude préalable et l'analyse des besoins. Le troisième chapitre détaille la phase de conception. Le quatrième chapitre décrit la réalisation technique des différentes composantes. Enfin, le cinquième chapitre est consacré aux tests, à la validation et au déploiement.

Chapitre

1

Cadre Général du Projet

Sommaire

1.1	Introduction	2
1.2	Contexte du projet	2
1.3	Présentation de l'organisme d'accueil	3
1.4	Problématique	3
1.5	Approche proposée	4
1.5.1	Fonctionnalités du CRM Web	4
1.5.2	Fonctionnalités de l'Application Mobile	5
1.5.3	Fonctionnalités du Chatbot IA	5
1.6	Méthodologie	5
1.6.1	Méthodologie Agile Scrum	5
1.6.2	Méthodologie CRISP-DM	6
1.6.3	Organisation en sprints	6
1.7	Architecture globale	7
1.8	Environnement technique	8
1.9	Conclusion	10

1.1 Introduction

Ce chapitre présente le cadre général du projet de stage. Il introduit le contexte métier dans lequel s'inscrit la réalisation du CRM, décrit l'organisme d'accueil, formalise la problématique rencontrée, expose l'approche retenue, précise la méthodologie adoptée et synthétise l'environnement technique mobilisé pour la mise en oeuvre du système.

1.2 Contexte du projet

La gestion de la relation client constitue aujourd'hui un facteur déterminant de compétitivité pour les entreprises de services numériques. Dans une structure comme **FRS**, l'activité quotidienne repose sur des échanges constants avec des prospects, des clients, des responsables métiers et des équipes techniques. Chaque interaction commerciale, chaque demande de support et chaque devis produit génèrent des informations qu'il devient nécessaire de centraliser afin d'éviter la perte de contexte et de renforcer la qualité de service.

Dans de nombreuses organisations, ces informations restent encore dispersées entre des feuilles de calcul, des boîtes email, des documents bureautiques et des échanges informels. Une telle situation ralentit la prise de décision, réduit la visibilité sur le pipeline commercial et complique la coordination entre les équipes.

Les principaux effets observés dans ce type d'organisation sont les suivants :

- difficulté à conserver une vision consolidée des prospects et des clients ;
- faible traçabilité des étapes commerciales et des relances effectuées ;
- manque de continuité entre le suivi commercial et le support après-vente ;
- absence d'espace client autonome pour consulter les tickets et les devis ;
- difficulté à exploiter rapidement la documentation métier et les procédures internes ;
- manque d'indicateurs opérationnels fiables pour piloter l'activité.

Impact d'une gestion dispersée de la relation client

Espace réservé à l'insertion d'une figure illustrant la dispersion des données entre fichiers Excel, emails, demandes de support et suivi commercial.

FIGURE 1.1 – Impact d'une gestion dispersée de la relation client

1.3 Présentation de l'organisme d'accueil

FRS (*FRS — IT Development Company*) est une société de services et d'ingénierie informatique installée à Tunis. L'entreprise intervient dans la conception et le développement de solutions logicielles sur mesure, l'intégration de services numériques, l'accompagnement applicatif et le support technique. Son positionnement repose sur une proximité métier avec ses clients et sur la capacité à proposer des solutions adaptées plutôt que des outils génériques difficilement maîtrisables.

Le projet de stage s'inscrit dans cette logique. Il vise à doter l'entreprise d'un système unifié de suivi commercial et de support, capable de servir à la fois les équipes internes et les clients externes, tout en restant suffisamment souple pour évoluer avec les besoins futurs.

TABLEAU 1.1 – Fiche signalétique de l'organisme d'accueil

Raison sociale	FRS — IT Development Company
Secteur d'activité	Services et ingénierie informatique, développement logiciel sur mesure
Siège	Tunis, Tunisie
Année de création	2018
Site web	www.frs.tn
Effectif	Environ 25 à 50 collaborateurs
Services principaux	Développement web et mobile, API et backend, solutions cloud, support applicatif, accompagnement numérique



FIGURE 1.2 – Logo de l'entreprise FRS

1.4 Problématique

L'analyse du contexte a mis en évidence un ensemble de besoins convergents. L'entreprise a besoin d'un outil unique pour structurer la prospection, transformer les opportunités commer-

ciales en dossiers clients exploitables, suivre les interactions de support et offrir aux clients un canal de consultation autonome.

La problématique peut être résumée ainsi : *comment concevoir une plateforme CRM cohérente, sécurisée et évolutive, capable de centraliser les données commerciales et support, tout en proposant une expérience adaptée aux utilisateurs internes et aux clients externes ?*

Plus précisément, les difficultés à résoudre sont les suivantes :

1. absence d'un référentiel unique pour les leads, comptes, contacts et opportunités ;
2. manque de continuité entre la qualification d'un prospect et sa transformation en client suivi ;
3. faible structuration du support client et de l'historique des échanges ;
4. absence de portail client pour la consultation des tickets et des devis ;
5. visibilité insuffisante sur les indicateurs de vente et de support ;
6. absence de journal d'audit pour tracer les actions sensibles ;
7. accès difficile à l'information métier et documentaire en situation de travail.

1.5 Approche proposée

Pour répondre à cette problématique, une approche en plusieurs composantes a été retenue. L'idée n'est pas de produire une simple interface de gestion, mais un écosystème cohérent articulant les besoins internes, l'accès client et l'assistance informationnelle.

TABLEAU 1.2 – Composantes fonctionnelles de la solution proposée

Composante	Rôle dans la solution
CRM web interne	Gestion des données commerciales et support par les équipes internes : leads, comptes, contacts, deals, devis, tickets, activités, utilisateurs, audit et reporting
Portail client web	Consultation sécurisée des tickets, messages, devis et informations de profil par les clients
Application mobile client	Accès nomade aux principales fonctions du portail client : tableau de bord, tickets, devis, profil et assistant conversationnel
Assistant conversationnel	Accès guidé à l'information métier à partir d'une base documentaire RAG regroupant présentation, services, FAQ, conditions générales, support et références projets

1.5.1 Fonctionnalités du CRM Web

Le CRM web constitue le coeur fonctionnel du projet. Il rassemble les modules nécessaires au pilotage commercial et au traitement du support dans une même application :

- authentification, gestion du profil et contrôle d'accès par rôles ;
- gestion des leads avec qualification et conversion ;

- gestion des comptes clients et des contacts associés ;
- pilotage du pipeline commercial et des devis ;
- gestion des tickets, des échanges publics et des notes internes ;
- suivi des activités commerciales et journalisation des actions ;
- tableaux de bord de vente et de support ;
- administration des utilisateurs et consultation de l’audit.

1.5.2 Fonctionnalités de l’Application Mobile

L’application mobile n’a pas vocation à reproduire l’ensemble du CRM interne. Elle est conçue comme une extension du portail client, orientée consultation et interaction :

- connexion sécurisée pour les utilisateurs de rôle client ;
- consultation du tableau de bord client ;
- suivi et création de tickets ;
- consultation des devis liés au compte client ;
- accès à un assistant conversationnel pour obtenir des informations documentaires ;
- consultation et mise à jour du profil.

1.5.3 Fonctionnalités du Chatbot IA

Le module d’assistance s’appuie sur une logique documentaire. Il ne remplace pas les modules CRM, mais vient compléter le système en facilitant l’accès aux informations métier :

- réponse en langage naturel à partir d’une base documentaire structurée ;
- interrogation de contenus liés à l’entreprise, aux services, à la FAQ et au support ;
- restitution d’une réponse contextualisée accompagnée de sources ;
- intégration dans le portail client web et dans l’application mobile ;
- fonctionnement dégradé possible lorsque le LLM externe n’est pas configuré.

1.6 Méthodologie

1.6.1 Méthodologie Agile Scrum

Le déroulement du projet a été structuré selon une logique **Agile Scrum**. Ce choix méthodologique permet d’avancer par itérations successives, de clarifier les objectifs à court terme et de réduire le risque de dérive entre le besoin exprimé et la solution développée. Chaque sprint est associé à un ensemble d’objectifs précis, depuis l’analyse jusqu’à la stabilisation finale du produit.

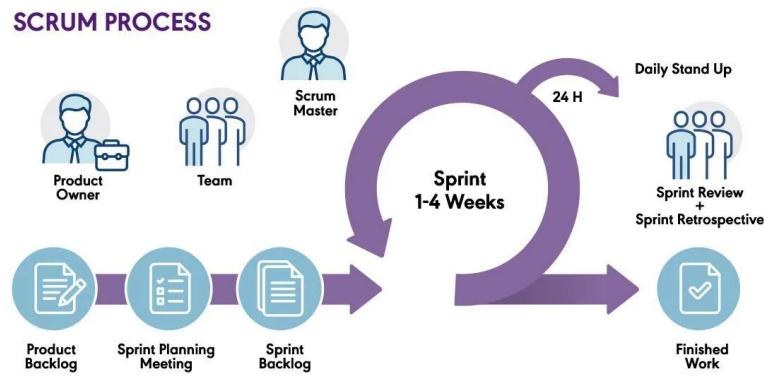


FIGURE 1.3 – Cycle général de la méthodologie Scrum

1.6.2 Méthodologie CRISP-DM

Pour le volet documentaire et conversationnel, une adaptation légère de la démarche **CRISP-DM** a été mobilisée. Elle a servi à structurer la sélection des documents, leur préparation, leur découpage, leur indexation et leur validation avant intégration dans la chaîne RAG. L'objectif n'était pas de réaliser un projet complet de data mining, mais d'utiliser une démarche méthodique pour constituer une base de connaissances exploitable et cohérente.

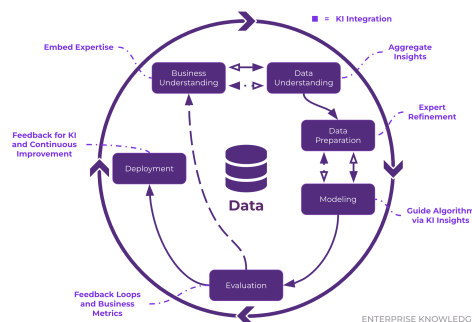


FIGURE 1.4 – Cycle CRISP-DM mobilisé pour la préparation de la base documentaire

1.6.3 Organisation en sprints

L'organisation temporelle du projet a été répartie en quatre sprints successifs, chacun associé à une finalité clairement identifiée.

TABLEAU 1.3 – Organisation du projet en sprints

Sprint	Finalité principale	Durée
Sprint 01	Étude préalable, analyse des besoins, identification des acteurs et élaboration du backlog	2 semaines
Sprint 02	Conception UML, modélisation des données, architecture technique et conception de l'API	2 semaines
Sprint 03	Réalisation du backend, du frontend, du portail client, du mobile et du module d'assistance	2 semaines
Sprint 04	Validation fonctionnelle, revue RBAC, corrections de stabilisation et préparation du déploiement local	2 semaines

1.7 Architecture globale

L'architecture retenue s'appuie sur une logique **trois tiers** enrichie par des composants périphériques. Les interfaces web et mobile communiquent avec une API centralisée exposée par le backend FastAPI. Cette API accède à la base relationnelle MySQL pour les données métier, sollicite une base vectorielle locale pour l'assistant documentaire, et peut déclencher des notifications email via SMTP. Le frontend web est servi par Nginx dans une configuration intégrée à Docker Compose.

Cette organisation présente plusieurs avantages :

- séparation claire entre présentation, logique métier et persistance ;
- réutilisation d'une même API par le web, le portail client et le mobile ;
- facilité de déploiement local grâce à la conteneurisation ;
- possibilité d'étendre ultérieurement le système par de nouveaux modules ou services.

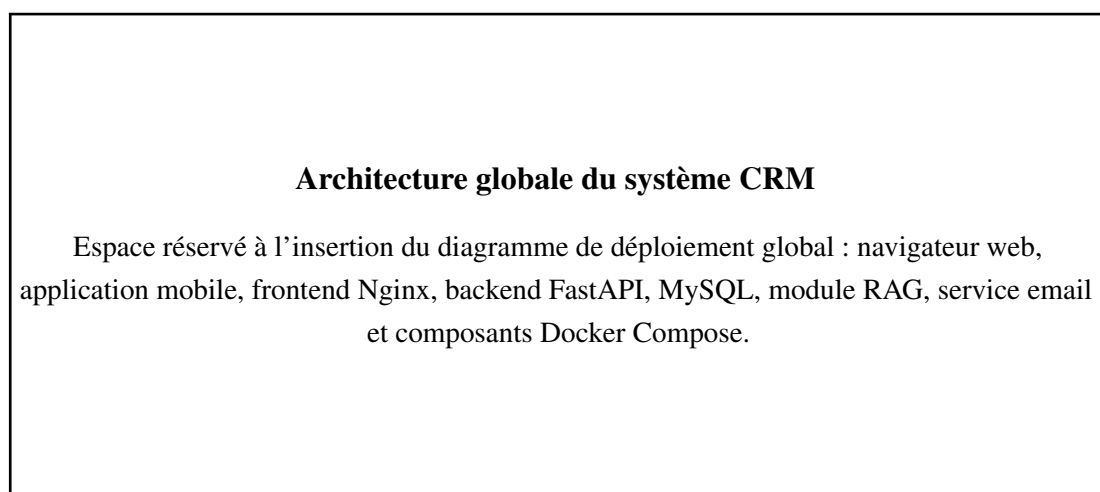


FIGURE 1.5 – Architecture globale du système CRM

La planification temporelle de notre projet est réalisée selon la méthode de Gantt, comme illustré dans le Tableau 1.4. Elle met en évidence les différentes phases du projet, à savoir l'analyse et le recueil des besoins, la conception UML et l'architecture, le développement des composantes, ainsi que les étapes de validation, de test et de stabilisation, jusqu'à la rédaction du rapport final.

TABLEAU 1.4 – Planification simplifiée du projet

Phase	S1	S2	S3	S4	S5	S6	S7	S8	
Analyse et recueil des besoins	•	•							
Conception UML et architecture			•	•					
Développement des composantes					•	•			
Validation, tests et stabilisation							•	•	
Rédaction du rapport							•	•	

1.8 Environnement technique

L'environnement technique du projet a été sélectionné de manière à concilier rapidité de développement, lisibilité du code et cohérence d'intégration entre les différentes couches du système.

TABLEAU 1.5 – Environnement technique du projet

Catégorie	Technologie	Utilisation dans le projet
Frontend web	React 19 + Vite	Construction de l'application SPA interne et du portail client web
Frontend web	Tailwind CSS + composants Radix UI	Mise en forme de l'interface, composants réutilisables et cohérence visuelle
Frontend web	Axios + i18next	Appels API, intercepteurs d'authentification et support multilingue FR/EN
Mobile	React Native + Expo	Développement de l'application mobile client multiplateforme
Mobile	React Navigation + Expo Secure Store	Navigation mobile et stockage sécurisé du jeton JWT
Backend	FastAPI	Exposition de l'API REST et structuration des routes métier
Backend	SQLAlchemy + Pydantic	Mapping ORM, validation des schémas et sérialisation des données
Sécurité	JWT + bcrypt/passlib	Authentification, vérification des mots de passe et contrôle d'accès RBAC
Base de données	MySQL 8.0	Stockage relationnel des données métier du CRM
Assistant documentaire	ChromaDB + LLM Groq optionnel	Indexation vectorielle locale et génération de réponses contextualisées
Infrastructure	Docker Compose + Nginx	Orchestration locale, reverse proxy frontend et exécution conteneurisée
IPSAS Tests	Pytest, Vitest, Jest	Validation backend, frontend et mobile
Outils de développement	Git + Visual Studio Code	Gestion de version et

1.9 Conclusion

Ce chapitre a permis de situer le projet dans son contexte métier, d'identifier la problématique à résoudre et de présenter les grandes lignes de la solution proposée. Il a également clarifié l'organisation méthodologique, l'architecture globale et l'environnement technique mobilisé. Ces éléments constituent la base de l'analyse détaillée menée dans le chapitre suivant.

Sprint 01

Étude Préalable et Analyse

Sprint 01 : Étude Préalable et Analyse

Sommaire

2.1	Introduction	13
2.2	Objectifs du Sprint	13
2.3	Étude de l'existant	13
2.3.1	Analyse des solutions CRM existantes	13
2.3.2	Critique de l'existant	14
2.4	Identification des acteurs	15
2.5	Analyse des besoins fonctionnels	15
2.5.1	Module Authentification et Sécurité	15
2.5.2	Module Leads (Prospects)	16
2.5.3	Module Accounts (Comptes Clients)	16
2.5.4	Module Contacts (Personnes)	16
2.5.5	Module Deals (Pipeline Commercial)	16
2.5.6	Module Quotes (Devis)	17
2.5.7	Module Tickets (Helpdesk)	17
2.5.8	Module Activities (Suivi Commercial)	17
2.5.9	Module Reporting et Audit	17
2.5.10	Module Portail Client	17
2.5.11	Module Application Mobile	18
2.5.12	Module Chatbot IA (RAG)	18
2.6	Besoins non fonctionnels	18
2.7	Modélisation UML — Diagrammes de cas d'utilisation	19
2.7.1	Diagramme de cas d'utilisation global	19
2.7.2	Cas d'utilisation — Gestion des Leads	21
2.7.3	Cas d'utilisation — Gestion des Tickets	21
2.7.4	Cas d'utilisation — Portail Client	22
2.7.5	Cas d'utilisation — Application Mobile	24
2.7.6	Cas d'utilisation — Chatbot IA	25
2.8	Backlog produit	25
2.9	Résultats du Sprint	26
2.10	Conclusion	26

2.1 Introduction

Le premier sprint a été consacré à la compréhension du besoin, à l'étude de l'existant et à la structuration fonctionnelle du futur système. Cette phase est essentielle, car elle conditionne la qualité des choix de conception et limite les risques d'écart entre la solution développée et les attentes métier.

2.2 Objectifs du Sprint

- analyser les limites des pratiques actuelles et des solutions CRM du marché ;
- identifier les acteurs impliqués dans le système ;
- formaliser les besoins fonctionnels et non fonctionnels ;
- modéliser les cas d'utilisation principaux ;
- établir un backlog produit pour planifier les réalisations.

2.3 Étude de l'existant

2.3.1 Analyse des solutions CRM existantes

L'étude préalable a porté à la fois sur les outils utilisés habituellement par les entreprises de services et sur les solutions CRM généralistes disponibles sur le marché. Les plateformes majeures proposent des fonctionnalités avancées, mais elles s'accompagnent souvent d'un coût élevé, d'une logique de paramétrage complexe et d'une dépendance forte à un écosystème externe.

Dans le cadre de **FRS**, l'objectif n'était pas de concurrencer ces plateformes sur tous les plans, mais de concevoir une solution ciblée, réellement adaptée au cycle de vente, au support et à la relation client de l'entreprise.

TABEAU 2.1 – Comparaison synthétique entre solutions existantes et solution développée

Critère	Salesforce	HubSpot	Solution développée
Coût d'adoption	Élevé à très élevé	Modèle freemium puis abonnements croissants	Maîtrisé par l'entreprise, adaptable au contexte de démonstration ou de déploiement local
Personnalisation métier	Forte mais souvent complexe	Moyenne selon les offres	Totale au niveau du code, des rôles et des flux métier
Portail client spécifique	Souvent optionnel ou additionnel	Partiel selon les modules	Intégré nativement au projet
Maîtrise des données	Hébergement externe	Hébergement externe	Déploiement local contenu possible
Intégration d'une assistance documentaire	Services additionnels	Variable selon l'offre	Module RAG intégré à la solution
Adéquation à un projet pédagogique ou expérimental	Faible	Moyenne	Très forte

2.3.2 Critique de l'existant

L'analyse a mis en évidence plusieurs limites dans les approches existantes ou dans les pratiques dispersées habituellement observées :

- centralisation insuffisante des informations commerciales ;
- difficulté à relier la prospection, la négociation et le support dans un même référentiel ;
- faible visibilité sur l'historique des interactions ;
- absence d'un portail autonome pour les clients ;
- coût élevé des solutions génériques lorsqu'un fort niveau de personnalisation est requis.

2.4 Identification des acteurs

Le système CRM fait intervenir cinq acteurs principaux. Chacun dispose d'un périmètre fonctionnel distinct, ce qui justifie la mise en place d'une gestion des rôles et des permissions côté serveur.

TABEAU 2.2 – Acteurs du système et responsabilités associées

Acteur	Rôle technique	Responsabilités principales
Administrateur	admin	Administration du système, création des utilisateurs, supervision globale, accès à l'audit et aux fonctions sensibles
Manager	manager	Supervision des équipes, lecture des rapports, suivi des indicateurs et contrôle des activités clés
Commercial	commercial	Prospection, qualification des leads, gestion des comptes, des contacts, des deals et des devis
Support	support	Traitement des tickets, réponses aux clients, suivi des échanges et gestion opérationnelle du helpdesk
Client	client	Accès au portail et au mobile pour consulter les devis, créer ou suivre les tickets, échanger avec le support et utiliser l'assistant documentaire

2.5 Analyse des besoins fonctionnels

Les besoins fonctionnels ont été regroupés par domaines métier afin de faciliter la future conception du système. Cette structuration reflète directement les modules observés dans le backend et les parcours disponibles dans les interfaces.

2.5.1 Module Authentification et Sécurité

Le système doit garantir un accès sécurisé aux interfaces et une séparation claire des droits d'utilisation :

- authentification par email et mot de passe ;
- émission d'un jeton JWT pour sécuriser les appels à l'API ;

- gestion des rôles admin, manager, commercial, support et client ;
- consultation et mise à jour du profil utilisateur ;
- redirection vers l'espace adapté selon le rôle.

2.5.2 Module Leads (Prospects)

Le module leads doit couvrir le cycle de qualification commerciale :

- création et mise à jour de fiches prospects ;
- suivi des statuts *new*, *contacted*, *qualified*, *unqualified* et *converted* ;
- filtrage par source, statut, responsable et recherche textuelle ;
- conversion d'un lead qualifié en compte, contact et opportunité commerciale.

2.5.3 Module Accounts (Comptes Clients)

Le compte client représente l'entité pivot des relations commerciales et support :

- centralisation des informations d'entreprise ;
- rattachement aux contacts, deals et tickets ;
- visibilité transversale pour les équipes commerciales et support ;
- administration des fiches comptes par les rôles internes.

2.5.4 Module Contacts (Personnes)

Le système doit permettre de gérer les interlocuteurs liés aux entreprises clientes :

- création et modification des contacts ;
- rattachement obligatoire à un compte ;
- recherche par identité, coordonnées et société liée ;
- conservation des informations de poste et de contexte métier.

2.5.5 Module Deals (Pipeline Commercial)

Le pipeline commercial doit fournir une vision opérationnelle des opportunités :

- gestion des opportunités par compte ;
- suivi des étapes *qualification*, *proposal*, *negotiation*, *won* et *lost* ;
- mise à jour des montants, probabilités et dates de clôture ;
- journalisation des changements de stage.

2.5.6 Module Quotes (Devis)

Le module devis doit permettre de cadrer les propositions commerciales :

- création de devis liés à un deal ;
- suivi des statuts *draft*, *sent*, *accepted* et *rejected* ;
- conservation de la référence, du montant et du support PDF éventuel ;
- notification du client lors de la création d'un nouveau devis.

2.5.7 Module Tickets (Helpdesk)

Le module support doit structurer le traitement des demandes clients :

- création de tickets par les équipes internes ou par les clients via le portail ;
- suivi des priorités et des catégories ;
- gestion des statuts *open*, *in_progress*, *waiting_customer*, *resolved* et *closed* ;
- messagerie associée avec distinction entre réponses publiques et notes internes ;
- assignation à des agents internes.

2.5.8 Module Activities (Suivi Commercial)

Le système doit conserver une trace des interactions commerciales significatives :

- enregistrement d'appels, d'emails, de réunions ou de notes ;
- liaison aux comptes, contacts ou deals ;
- affichage chronologique des actions effectuées ;
- exploitation dans une logique de relance et de suivi commercial.

2.5.9 Module Reporting et Audit

Le projet doit offrir des fonctions d'analyse et de traçabilité :

- indicateurs de vente : répartition des leads, pipeline, valeur gagnée, taux de conversion ;
- indicateurs support : tickets ouverts, tickets en retard, temps moyen de résolution ;
- audit des opérations sensibles pour conserver un historique exploitable.

2.5.10 Module Portail Client

Le portail client répond à un besoin fort d'autonomie et de transparence :

- consultation des tickets et de leurs réponses publiques ;
- création de tickets depuis l'espace client ;
- consultation des devis liés au compte ;
- accès aux informations de profil ;
- isolement strict des données par compte client.

2.5.11 Module Application Mobile

L'application mobile reprend l'essentiel du portail client dans une logique d'usage nomade :

- authentification client sécurisée ;
- tableau de bord synthétique ;
- suivi des tickets et ajout de messages ;
- consultation des devis ;
- accès à l'assistant documentaire ;
- message de blocage pour les rôles internes non prévus sur mobile.

2.5.12 Module Chatbot IA (RAG)

Le module d'assistance a pour objectif de faciliter l'accès à l'information métier et documentaire :

- interrogation en langage naturel de documents d'entreprise ;
- restitution d'une réponse synthétique et contextualisée ;
- affichage des sources utilisées pour construire la réponse ;
- fonctionnement même en mode dégradé lorsque le fournisseur LLM n'est pas disponible.

2.6 Besoins non fonctionnels

Au-delà des fonctionnalités, le système doit respecter plusieurs exigences transverses qui influencent directement les choix techniques.

TABLEAU 2.3 – Principaux besoins non fonctionnels

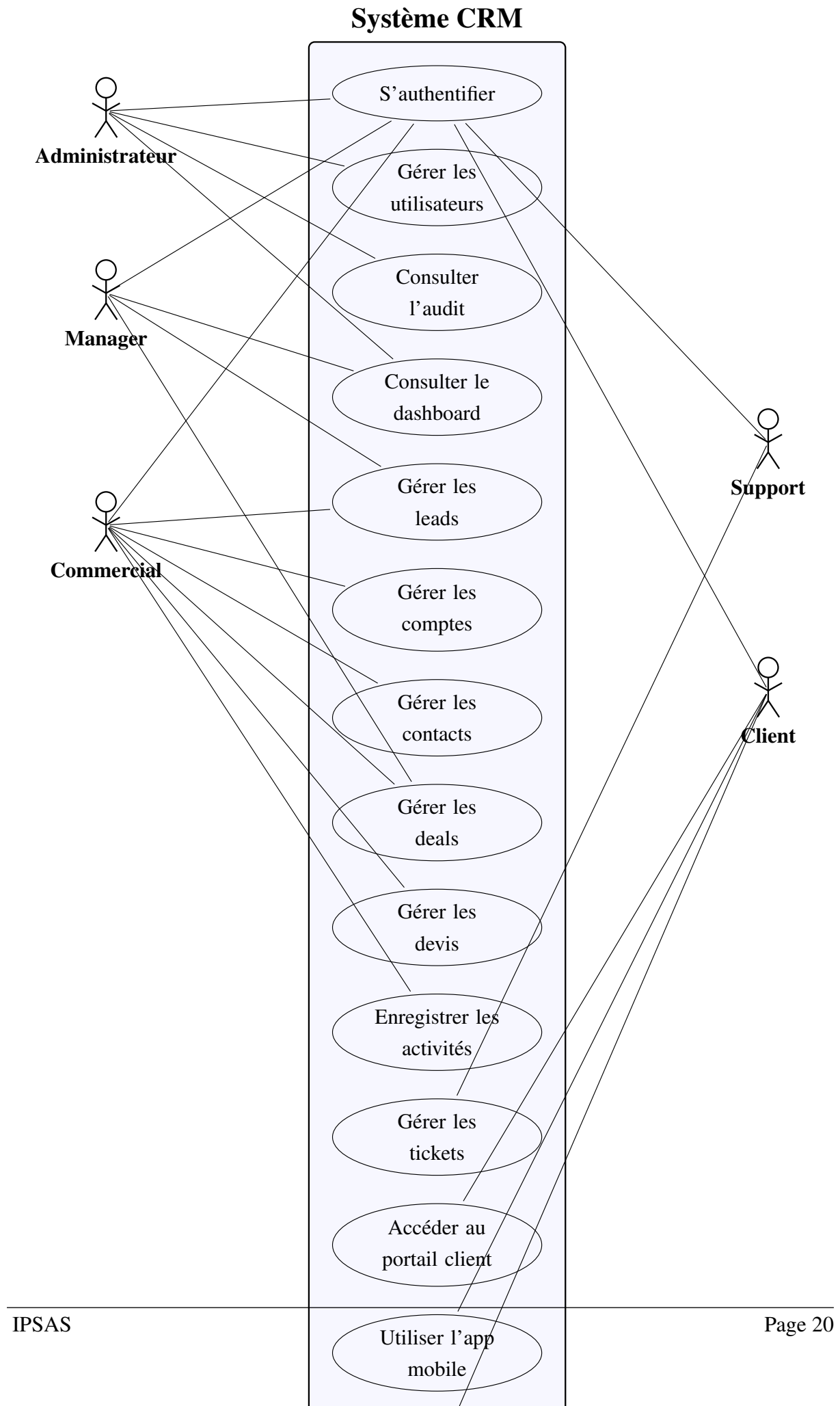
Exigence	Description attendue
Sécurité	Authentification robuste, contrôle d'accès par rôles, filtrage serveur des données visibles par chaque profil
Maintenabilité	Structuration modulaire du code, séparation des responsabilités et lisibilité des composants métier
Traçabilité	Journalisation des actions importantes pour faciliter la supervision et l'analyse postérieure
Portabilité	Possibilité d'exécuter localement le système et ses dépendances via Docker Compose
Ergonomie	Interfaces claires pour les usages internes et clients, adaptées au web et au mobile
Internationalisation	Support de plusieurs langues sur les interfaces web, avec une base FR/EN existante
Évolutivité	Capacité à ajouter de nouveaux modules, écrans ou sources documentaires sans remise à plat de l'architecture

2.7 Modélisation UML — Diagrammes de cas d'utilisation

Les diagrammes de cas d'utilisation permettent de relier les acteurs identifiés aux services attendus du système. Ils jouent un rôle important dans la clarification du périmètre fonctionnel avant le passage à la conception détaillée.

2.7.1 Diagramme de cas d'utilisation global

Le diagramme global représente la vision d'ensemble du système avec ses cinq acteurs et ses principaux blocs fonctionnels. Chaque acteur est relié uniquement aux cas d'utilisation relevant de son périmètre de responsabilité.



2.7.2 Cas d'utilisation — Gestion des Leads

Ce diagramme met en évidence le cycle d'acquisition commerciale, de la création d'un lead jusqu'à sa conversion en compte, contact et deal.

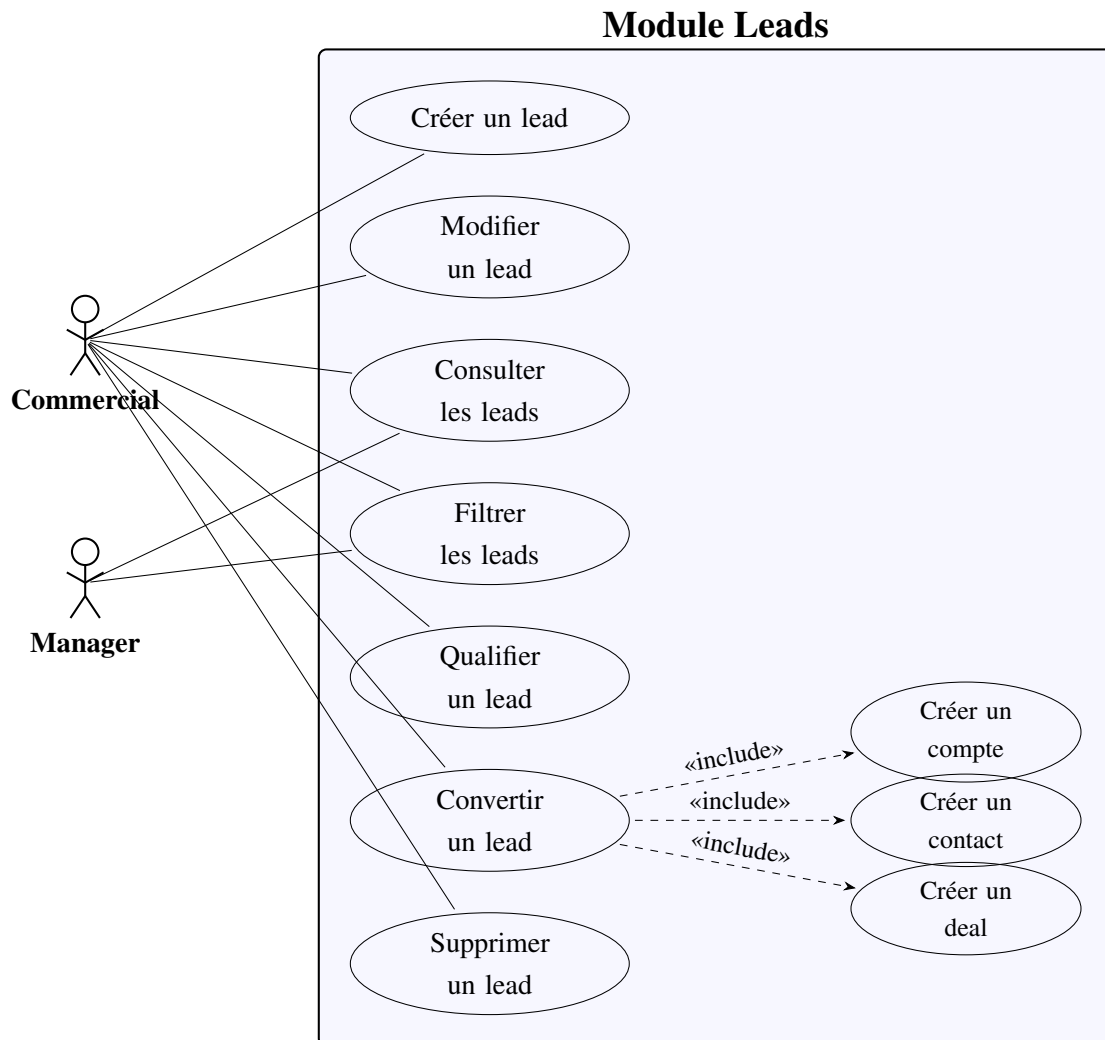


FIGURE 2.2 – Cas d'utilisation du module Leads

2.7.3 Cas d'utilisation — Gestion des Tickets

Le module support fait intervenir plusieurs interactions entre le client, les agents internes et les responsables de supervision.

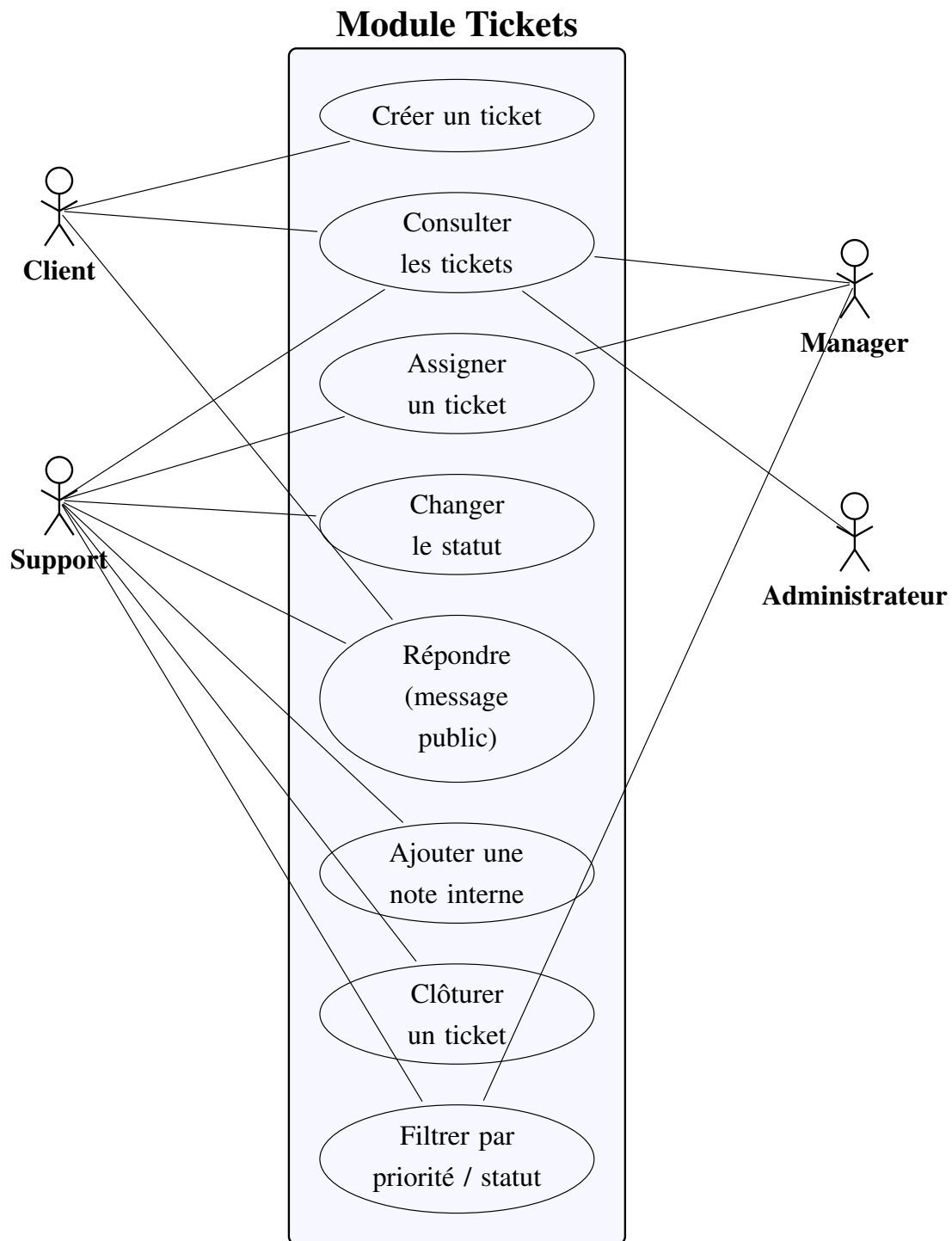
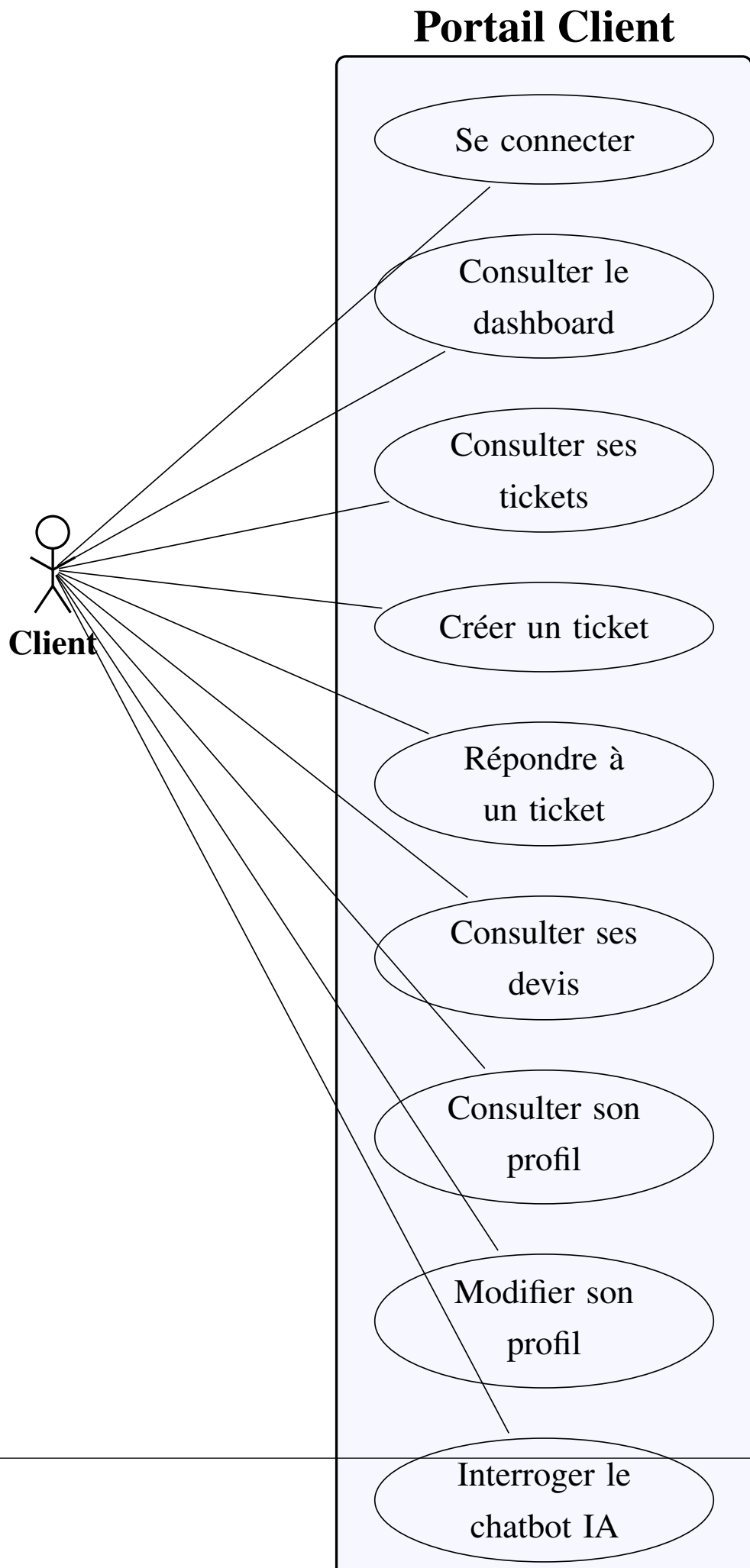


FIGURE 2.3 – Cas d'utilisation du module Tickets

2.7.4 Cas d'utilisation — Portail Client

Le portail client reste volontairement ciblé sur l'autonomie, la consultation et la communication avec le support.



2.7.5 Cas d'utilisation — Application Mobile

L'application mobile prolonge les usages du portail client dans un contexte d'accès nomade. Les rôles internes reçoivent un message de blocage les invitant à utiliser l'interface web.

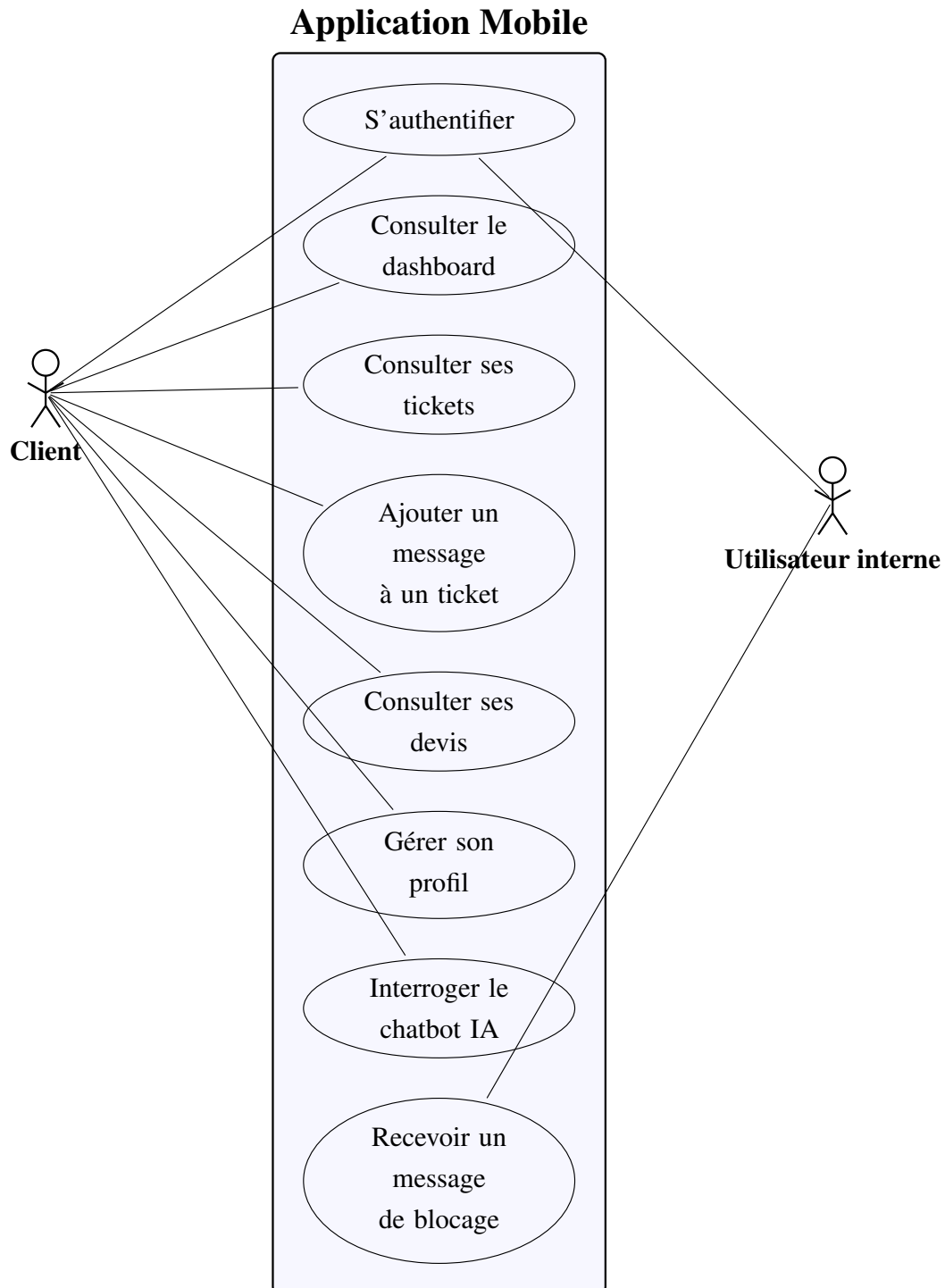


FIGURE 2.5 – Cas d'utilisation de l'application mobile

2.7.6 Cas d'utilisation — Chatbot IA

Le module d'assistance peut être représenté comme un service transverse permettant à l'utilisateur authentifié d'interroger la base documentaire et d'obtenir une réponse contextualisée.

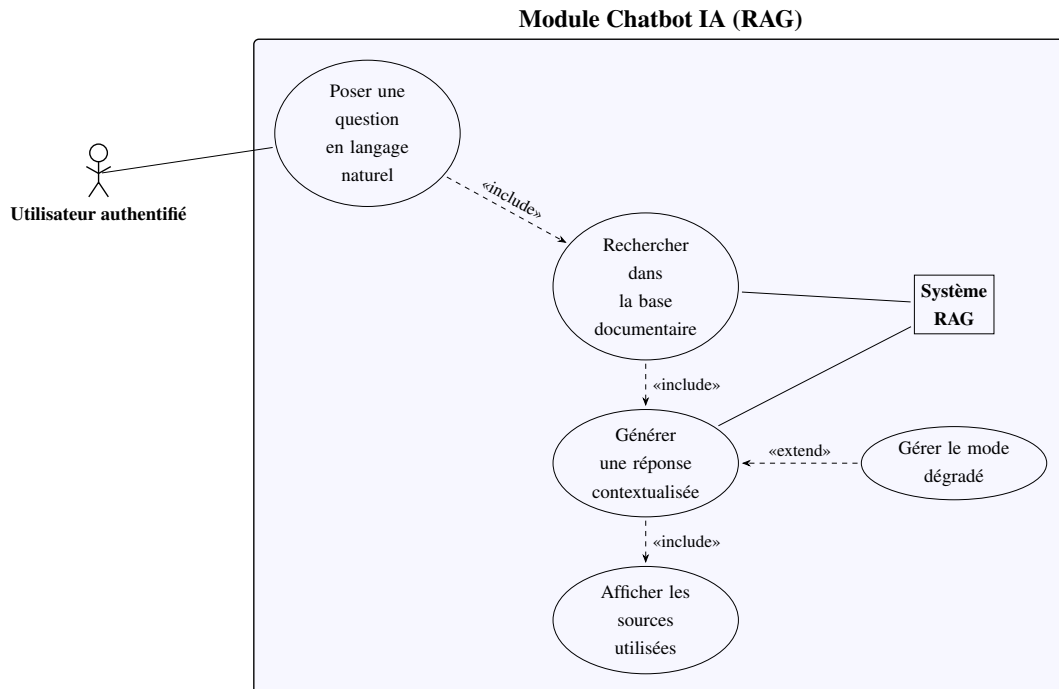


FIGURE 2.6 – Cas d'utilisation du chatbot documentaire (RAG)

2.8 Backlog produit

Le backlog suivant synthétise les besoins prioritaires identifiés au terme de l'analyse. Il a servi de base à la planification des sprints techniques.

TABLEAU 2.4 – Backlog produit synthétique

ID	User Story	Priorité	Sprint cible
US01	En tant que commercial, je veux créer, qualifier et suivre un lead afin d'organiser la prospection	Haute	Sprint 03
US02	En tant que commercial, je veux convertir un lead qualifié en compte, contact et deal afin de réduire la ressaisie	Haute	Sprint 03
US03	En tant que commercial ou manager, je veux piloter le pipeline des deals pour suivre la maturité des opportunités	Haute	Sprint 03
US04	En tant que support, je veux gérer des tickets avec réponses publiques et notes internes afin de structurer le helpdesk	Haute	Sprint 03
US05	En tant que client, je veux consulter mes tickets et mes devis dans un portail dédié afin de gagner en autonomie	Haute	Sprint 03
US06	En tant qu'administrateur, je veux gérer les utilisateurs et les rôles afin de sécuriser l'accès au système	Haute	Sprint 03
US07	En tant que manager, je veux consulter des indicateurs de vente et de support afin de piloter l'activité	Haute	Sprint 03
US08	En tant qu'utilisateur interne, je veux enregistrer des activités commerciales afin de conserver l'historique des interactions	Moyenne	Sprint 03
US09	En tant que client, je veux accéder à mon espace depuis un mobile afin de suivre mes demandes en mobilité	Moyenne	Sprint 03
US10	En tant qu'utilisateur authentifié, je veux interroger la base documentaire via un assistant conversationnel afin d'obtenir rapidement une information métier	Moyenne	Sprint 03

2.9 Résultats du Sprint

TABLEAU 2.5 – Livrables du Sprint 01

Livrable	Statut
Étude comparative et critique de l'existant	✓
Identification des acteurs et des responsabilités	✓
Formalisation des besoins fonctionnels et non fonctionnels	✓
Préparation des cas d'utilisation du système	✓
Élaboration d'un backlog produit structuré	✓

2.10 Conclusion

Le premier sprint a permis de poser les fondations du projet. L'étude de l'existant a justifié la pertinence d'une solution sur mesure, l'identification des acteurs a clarifié le périmètre d'utilisation, et l'analyse des besoins a fourni un cadre précis pour la conception détaillée. Le chapitre suivant s'appuie sur ces résultats pour présenter les modèles, les diagrammes et l'architecture de la solution.

Sprint 02

Conception du Système CRM

Sprint 02 : Conception du Système CRM

Sommaire

3.1	Introduction	29
3.2	Objectifs du Sprint	29
3.3	Diagramme de classes	29
3.3.1	Relations entre entités	30
3.4	Diagramme Entité-Relation (ERD)	31
3.5	Diagrammes de séquence	31
3.5.1	Séquence — Authentification JWT	31
3.5.2	Séquence — Conversion d'un Lead	32
3.5.3	Séquence — Cycle de vie d'un Ticket	32
3.5.4	Séquence — Chatbot documentaire (RAG)	33
3.6	Conception de l'architecture	33
3.6.1	Architecture backend — FastAPI + organisation par domaines	33
3.6.2	Architecture frontend — React SPA	34
3.6.3	Architecture mobile — React Native	34
3.6.4	Architecture RAG — Chatbot IA	35
3.6.5	Architecture Docker Compose	35
3.7	Conception de l'API REST	36
3.8	Résultats du Sprint	37
3.9	Conclusion	37

3.1 Introduction

Ce chapitre présente le deuxième sprint, consacré à la conception détaillée du système. À partir des besoins identifiés précédemment, il s'agit ici de structurer les modèles métier, de préciser le schéma de données, de décrire les flux critiques du système et de définir une architecture capable de supporter les interfaces web, client, mobile et documentaire.

3.2 Objectifs du Sprint

- modéliser les entités métier et leurs relations ;
- élaborer le schéma relationnel de la base de données ;
- formaliser les séquences de fonctionnement essentielles ;
- définir l'architecture technique du backend, du frontend, du mobile et du module RAG ;
- spécifier les principaux endpoints de l'API REST.

3.3 Diagramme de classes

Le diagramme de classes permet de représenter les objets métiers manipulés par le système ainsi que leurs liens. Dans le cas du CRM réalisé, la modélisation tourne autour de deux grands axes : le cycle commercial et le cycle de support. À ces deux axes s'ajoutent des éléments transverses comme l'utilisateur, les activités, l'audit et les rôles.

TABEAU 3.1 – Entités principales du diagramme de classes

Classe métier	Rôle dans le système
User	Représente un utilisateur authentifié du système, interne ou client, avec un rôle et un périmètre d'accès
Account	Porte les informations de l'entreprise cliente et sert d'entité pivot pour les contacts, deals et tickets
Contact	Représente l'interlocuteur rattaché à un compte client
Lead	Représente un prospect avant sa conversion en éléments structurés du CRM
Deal	Représente une opportunité commerciale suivie dans le pipeline
Quote	Représente un devis associé à une opportunité commerciale
Ticket	Représente une demande de support liée à un compte client
TicketMessage	Représente un message public ou une note interne dans le cycle de vie d'un ticket
Activity	Représente une interaction commerciale liée à un compte, un contact ou un deal
AuditLog	Conserve l'historique des opérations sensibles pour assurer la traçabilité

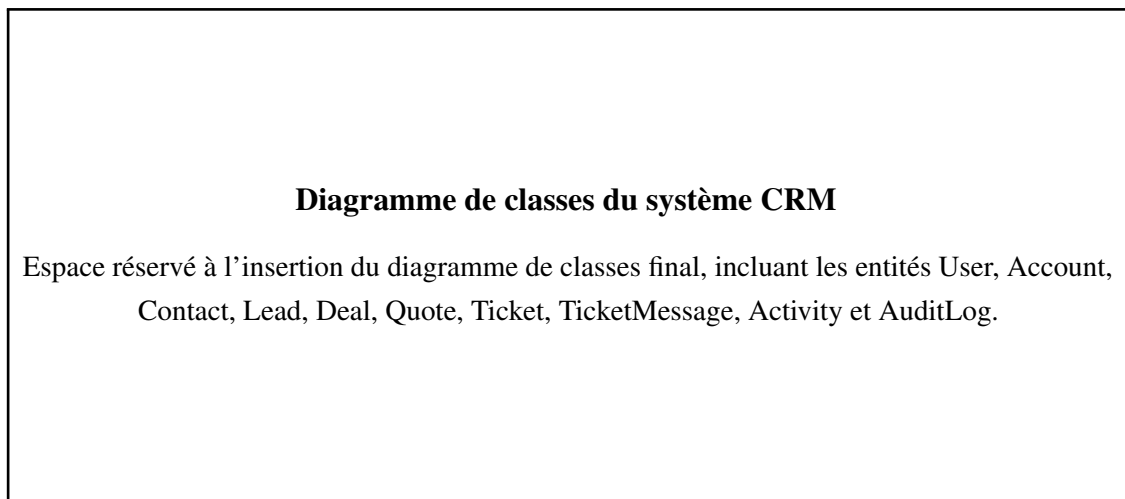


FIGURE 3.1 – Diagramme de classes du système CRM

3.3.1 Relations entre entités

Les relations entre classes structurent le comportement global du système. Les plus importantes sont résumées ci-dessous.

TABLEAU 3.2 – Principales relations métier du système

Relation	Interprétation métier
Account — Contact	Un compte regroupe plusieurs interlocuteurs métiers
Account — Deal	Une entreprise cliente peut faire l'objet de plusieurs opportunités commerciales
Deal — Quote	Une opportunité peut donner lieu à plusieurs versions ou états de devis
Account — Ticket	Les demandes de support sont rattachées à l'entreprise cliente concernée
Ticket — TicketMessage	Chaque ticket conserve l'historique chronologique des échanges associés
Lead — Account / Contact / Deal	La conversion d'un lead crée les entités commerciales structurées et mémorise les références correspondantes
User — Account	Un utilisateur client peut être lié à un compte; un utilisateur interne peut être propriétaire d'un compte, d'un deal ou d'un lead
Activity — Account / Contact / Deal	Une activité commerciale peut être liée à une ou plusieurs entités métier suivies
AuditLog — User	Chaque action auditée est rattachée, quand cela est possible, à son acteur

3.4 Diagramme Entité-Relation (ERD)

Le schéma de base de données traduit le modèle objet en structure relationnelle. Le projet s'appuie sur une base **MySQL** contenant dix tables principales. Ces tables couvrent l'authentification, les objets métiers du cycle commercial, le helpdesk et l'audit. Les relations sont implémentées par des clés étrangères assurant la cohérence entre les comptes, les contacts, les deals, les devis, les tickets et les utilisateurs.

La conception du schéma a été pensée pour répondre à deux exigences : d'une part, conserver des relations simples à comprendre et à exploiter côté ORM ; d'autre part, permettre l'évolution des modules sans remettre en cause la structure générale du système.

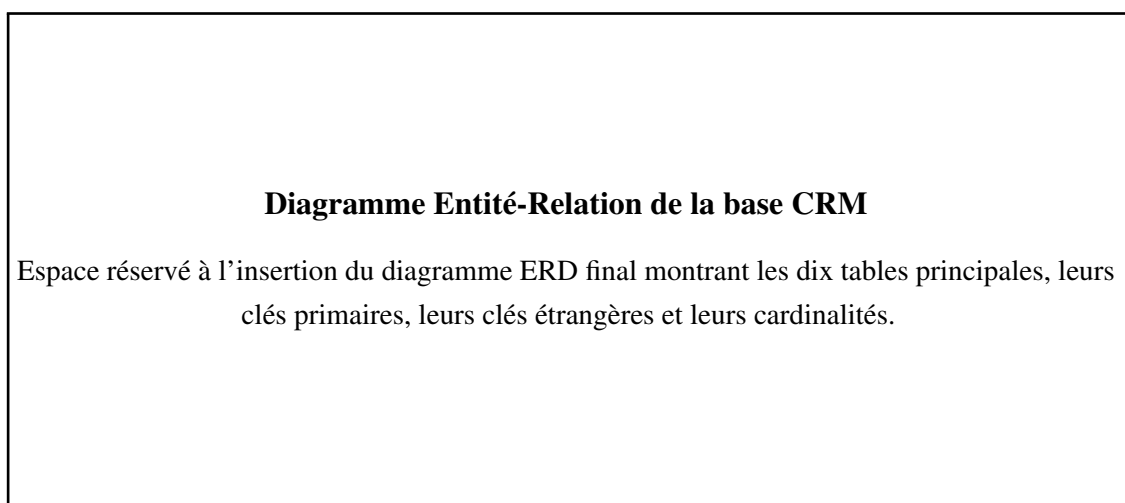


FIGURE 3.2 – Diagramme Entité-Relation de la base CRM

3.5 Diagrammes de séquence

Les diagrammes de séquence décrivent les interactions dynamiques entre les interfaces, l'API et les couches métier du backend. Ils mettent en évidence les étapes de traitement des scénarios critiques.

3.5.1 Séquence — Authentification JWT

Le scénario d'authentification repose sur la soumission des identifiants, la vérification du mot de passe, la génération d'un jeton JWT et la récupération du profil courant. Ce flux conditionne l'accès à tous les autres modules du système.

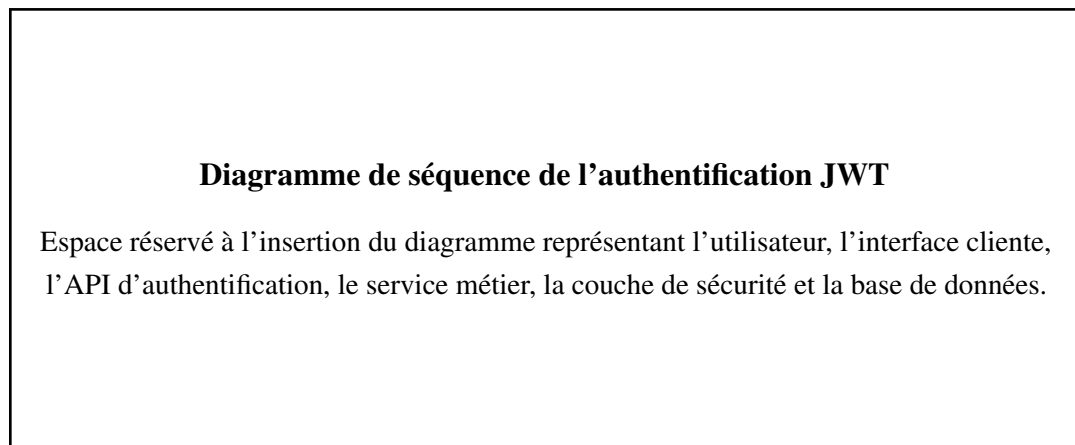


FIGURE 3.3 – Diagramme de séquence de l'authentification JWT

3.5.2 Séquence — Conversion d'un Lead

La conversion d'un lead est l'un des scénarios métier les plus importants. Elle illustre la manière dont une opération unique aboutit à la création de plusieurs entités corrélées : compte, contact et deal. Ce flux doit rester transactionnel et cohérent.

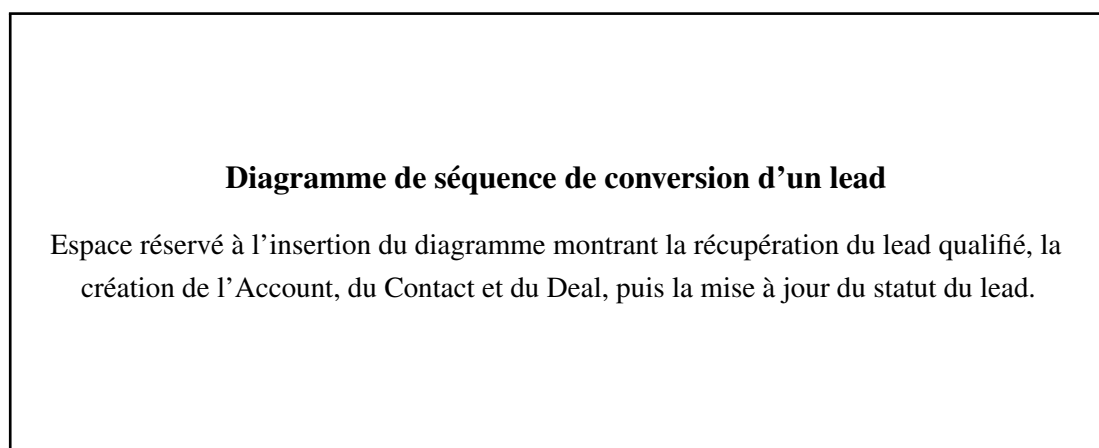


FIGURE 3.4 – Diagramme de séquence de conversion d'un lead

3.5.3 Séquence — Cycle de vie d'un Ticket

Le traitement d'un ticket implique plusieurs participants : le client, le portail ou l'espace interne, l'API support, les services métier, la base de données et le système de notification. La séquence doit montrer la distinction entre message public et note interne ainsi que la mise à jour du statut du ticket.

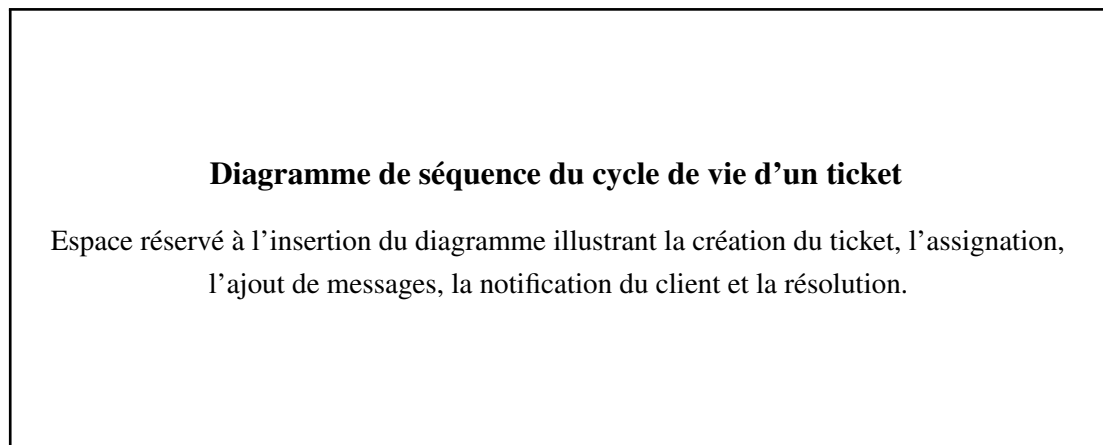


FIGURE 3.5 – Diagramme de séquence du cycle de vie d'un ticket

3.5.4 Séquence — Chatbot documentaire (RAG)

Le flux du module d'assistance repose sur une logique documentaire : l'utilisateur soumet une question, le backend recherche les fragments les plus pertinents dans la base vectorielle, puis renvoie une réponse contextualisée avec ses sources. Le LLM externe intervient uniquement lorsqu'il est configuré ; sinon, le système retourne une réponse de secours basée sur la recherche locale.

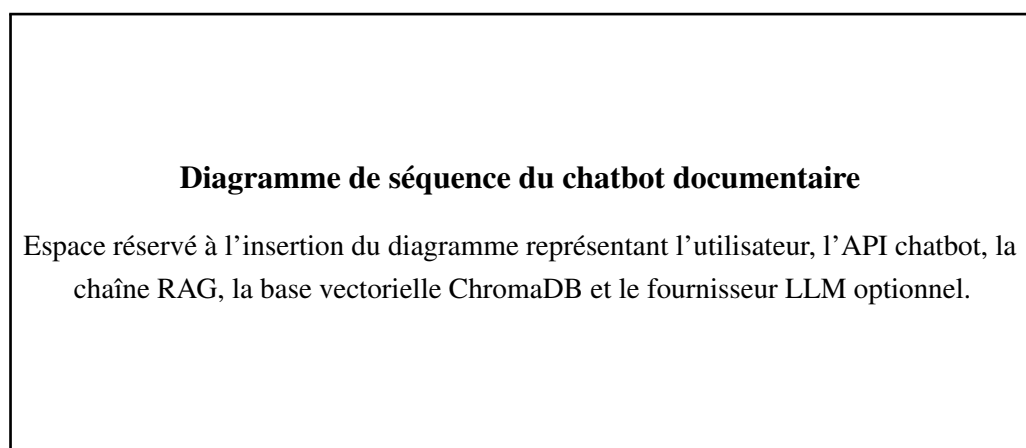


FIGURE 3.6 – Diagramme de séquence du chatbot documentaire

3.6 Conception de l'architecture

3.6.1 Architecture backend — FastAPI + organisation par domaines

Le backend est conçu comme une API REST structurée par domaines métier. Chaque domaine rassemble les modèles, schémas, repositories et services correspondant à une responsabilité fonctionnelle précise. Cette organisation facilite la lisibilité, limite le couplage et rend les évolutions plus prévisibles.

TABLEAU 3.3 – Organisation logique du backend

Couche	Responsabilité	Exemples dans le projet
API	Exposer les endpoints, valider les dépendances et orchestrer les services	routeurs auth, leads, tickets, client, chatbot, email
Domaines métier	Encapsuler les règles métier et la logique applicative	services de conversion de lead, gestion des tickets, reporting, audit
Persistance	Accéder à la base relationnelle via l'ORM	repositories SQLAlchemy et modèles métier
Composants partagés	Fournir sécurité, pagination, exceptions et services transverses	JWT, pagination, email, base repository
Composants documentaires	Gérer l'indexation, la recherche et la génération de réponse documentaire	vector store, chaîne RAG, client LLM

3.6.2 Architecture frontend — React SPA

Le frontend web est pensé comme une application SPA séparant les parcours internes et les parcours clients. L'espace interne s'appuie sur un layout dédié pour les modules CRM, tandis que le portail client dispose de son propre layout et d'une logique de navigation distincte. La communication avec le backend est centralisée dans un service Axios, complété par un contexte d'authentification et un contexte de thème.

Cette séparation apporte plusieurs avantages :

- clarté des parcours selon les profils ;
- réutilisation d'un même backend pour plusieurs interfaces ;
- mise en place de comportements transverses comme l'authentification, l'internationalisation et le mode sombre ;
- intégration d'un widget de chat documentaire dans l'espace client.

3.6.3 Architecture mobile — React Native

L'application mobile reprend la logique du portail client dans un contexte natif léger. Elle repose sur une navigation de type **Native Stack**, sur un contexte d'authentification et sur un stockage sécurisé du jeton via **Expo Secure Store**. Le choix d'une application dédiée au rôle client simplifie les flux et limite les risques d'incohérence avec les parcours internes.

TABLEAU 3.4 – Principes de conception de l'application mobile

Bloc	Description
Navigation	Enchaînement des écrans login, tableau de bord client, tickets, devis, chatbot et profil
Authentification	Gestion du jeton et récupération du profil via un contexte React Native
Sécurité locale	Stockage du jeton JWT dans Expo Secure Store
Services	Centralisation des appels API via Axios
Restriction métier	Blocage explicite des rôles internes, redirigés vers la version web

3.6.4 Architecture RAG — Chatbot IA

Le module d'assistance adopte une architecture documentaire simple et robuste :

1. préparation d'un ensemble de documents métier rédigés en Markdown ;
2. découpage de ces documents en fragments exploitables ;
3. indexation de ces fragments dans une base vectorielle locale ;
4. recherche des passages les plus pertinents pour une question donnée ;
5. génération d'une réponse contextualisée à partir des résultats obtenus ;
6. repli sur une réponse documentaire simplifiée lorsque le LLM externe n'est pas disponible.

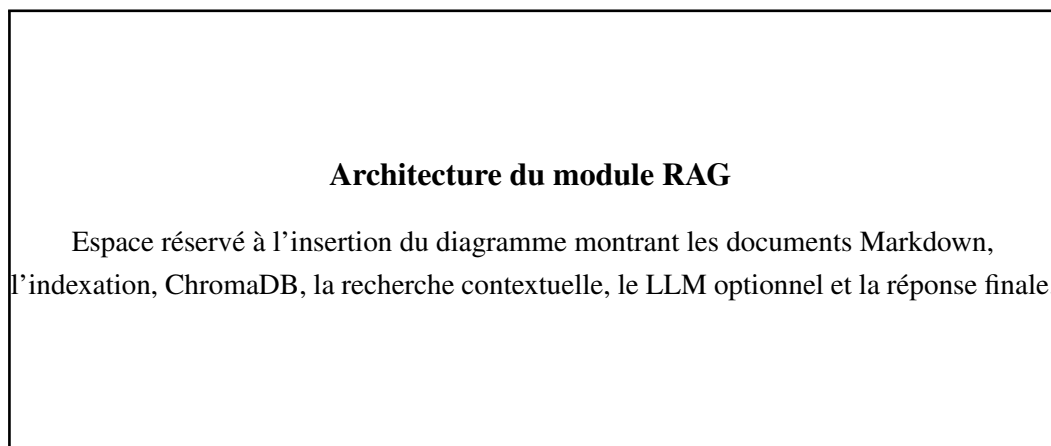


FIGURE 3.7 – Architecture du module RAG

3.6.5 Architecture Docker Compose

Le projet est conçu pour être exécuté localement dans un environnement conteneurisé. La pile principale comprend une base de données MySQL, un backend FastAPI et un frontend servi par Nginx. Des services supplémentaires dédiés aux campagnes de test sont définis dans le fichier `docker-compose.yml`.

TABLEAU 3.5 – Services définis dans l’architecture Docker Compose

Service	Port principal	Rôle
db	3307 sur l’hôte	Base MySQL du projet
backend	8000	API FastAPI et logique métier
frontend	3000	Application web servie via Nginx avec reverse proxy API
backend_unit_tests	—	Exécution des tests unitaires backend
backend_integration_tests	—	Exécution des tests d’intégration API
frontend_tests	—	Exécution des tests frontend
mobile_tests	—	Exécution des tests mobile

3.7 Conception de l’API REST

La conception de l’API REST s’appuie sur une logique de routes métier versionnées en /api/v1. Les endpoints couvrent l’authentification, la gestion des entités commerciales, le help-desk, le portail client, les rapports, les notifications email et le module d’assistance.

TABLEAU 3.6 – Extrait des principaux endpoints de l’API REST

Méthode	Endpoint	Description
POST	/api/v1/auth/login	Authentification et retour d’un jeton JWT
GET	/api/v1/auth/me	Récupération du profil utilisateur connecté
GET / POST	/api/v1/leads	Consultation et création des leads
POST	/api/v1/leads/{id}/convert	Conversion d’un lead qualifié en compte, contact et deal
GET / POST	/api/v1/deals	Consultation et création des opportunités commerciales
PATCH	/api/v1/deals/{id}/stage	Mise à jour du stage d’un deal dans le pipeline
GET / POST	/api/v1/quotes	Consultation et création des devis
GET / POST	/api/v1/tickets/{id}/messages	Consultation et ajout de messages sur un ticket interne
GET	/api/v1/reports/sales	Indicateurs de performance commerciale
GET	/api/v1/reports/support	Indicateurs du support client
GET	/api/v1/reports/audit-logs	Consultation du journal d’audit
GET / POST	/api/v1/client/tickets	Consultation et création de tickets côté client
GET	/api/v1/client/quotes	Consultation des devis liés au compte client
POST	/api/v1/chatbot	Envoi d’une question au module d’assistance documentaire
GET	/api/v1/chatbot/status	Consultation de l’état du module RAG
GET / POST	/api/v1/email/...	Vérification et envoi de notifications email

3.8 Résultats du Sprint

TABLEAU 3.7 – Livrables du Sprint 02

Livrable	Statut
Modèle de classes métier et relations principales	✓
Schéma relationnel et principes ERD	✓
Séquences des scénarios critiques	✓
Architecture technique backend, frontend, mobile et RAG	✓
Spécification synthétique de l'API REST	✓

3.9 Conclusion

Le deuxième sprint a permis de transformer les besoins du chapitre précédent en une vision structurée de la solution. Les modèles de données, les flux dynamiques et l'architecture technique sont désormais clarifiés. Cette conception sert de référence pour la phase suivante, consacrée à la réalisation concrète des différentes composantes du système.

Sprint 03

Réalisation Technique

Chapitre

4

Sprint 03 : Réalisation Technique

Sommaire

4.1	Introduction	40
4.2	Objectifs du Sprint	40
4.3	Réalisation du Backend (FastAPI)	40
4.3.1	Authentification JWT et RBAC	40
4.3.2	Logique de conversion Lead	41
4.3.3	Système de messagerie Ticket	41
4.3.4	Journal d'audit et reporting	41
4.3.5	Notifications email	42
4.4	Réalisation du Frontend (React)	42
4.4.1	Pages principales	42
4.4.2	Portail Client	44
4.4.3	Fonctionnalités UX avancées	44
4.5	Réalisation de l'Application Mobile (React Native)	45
4.5.1	Technologies utilisées	45
4.5.2	Écrans de l'application	45
4.6	Réalisation du Chatbot IA (RAG)	46
4.6.1	Architecture RAG	46
4.6.2	Implémentation	46
4.6.3	Documents RAG	47
4.7	Déploiement Docker	47
4.8	Résultats du Sprint	48
4.9	Conclusion	48

4.1 Introduction

Ce chapitre présente la phase de réalisation technique du projet. Après la conception détaillée menée au sprint précédent, l'objectif de ce sprint est de transformer les modèles et les choix d'architecture en composants opérationnels : backend, interfaces web, portail client, application mobile, assistant documentaire et infrastructure d'exécution locale.

4.2 Objectifs du Sprint

- implémenter le backend FastAPI et les modules métier du CRM ;
- développer l'interface web interne et le portail client ;
- réaliser l'application mobile client avec React Native ;
- intégrer le module d'assistance documentaire basé sur RAG ;
- préparer l'exécution locale conteneurisée de l'ensemble.

4.3 Réalisation du Backend (FastAPI)

Le backend constitue la colonne vertébrale du système. Il concentre la logique métier, les contrôles d'accès, l'accès aux données, les mécanismes de reporting, l'audit et les composants transverses tels que les notifications email et l'assistance documentaire.

4.3.1 Authentification JWT et RBAC

L'authentification repose sur un flux simple et sécurisé : l'utilisateur soumet ses identifiants, l'API vérifie le mot de passe chiffré, puis génère un jeton JWT utilisé pour les requêtes suivantes. Cette logique est complétée par une gestion fine des permissions, avec cinq rôles distincts (admin, manager, commercial, support, client).

Le contrôle d'accès est centralisé dans des dépendances FastAPI réutilisées par les routeurs. Ce choix permet de sécuriser les endpoints sans dupliquer la logique d'autorisation dans chaque module. Il facilite également la maintenance de la politique RBAC à mesure que le projet évolue.

TABLEAU 4.1 – Principes de réalisation du module d’authentification

Composant	Rôle
JWT	Authentification stateless pour les clients web et mobile
bcrypt / passlib	Hachage et vérification sécurisée des mots de passe
Dependencies FastAPI	Contrôle d’accès par rôles et filtrage des utilisateurs internes ou clients
AuthService	Centralisation de la logique de connexion, profil et gestion des utilisateurs

4.3.2 Logique de conversion Lead

La conversion d’un lead est l’un des mécanismes métier les plus sensibles du CRM. Lorsqu’un prospect atteint l’état *qualified*, le service de conversion crée un compte client, un contact et un deal dans une même séquence de traitement. Le lead est ensuite marqué comme *converted* et conserve la référence vers les objets générés.

Cette logique répond à un besoin important : éviter la ressaisie des données et assurer une continuité fonctionnelle entre prospection et suivi commercial. Le backend a donc été conçu pour porter explicitement cette transition métier dans un service dédié plutôt que de laisser l’utilisateur reconstruire manuellement les informations sur plusieurs écrans.

4.3.3 Système de messagerie Ticket

Le module ticketing prend en charge deux types d’échanges : les réponses publiques, visibles par le client, et les notes internes, réservées aux agents. Cette séparation est importante pour préserver un espace de coordination interne sans exposer au client des éléments techniques ou organisationnels non pertinents.

Le backend gère également la relation entre tickets, comptes, contacts et utilisateurs assignés. Le portail client applique un filtrage strict sur le périmètre du compte connecté et n’expose jamais les notes internes. L’ensemble permet de maintenir une cohérence entre espace interne et espace client tout en respectant les règles de visibilité.

4.3.4 Journal d’audit et reporting

Les opérations critiques du système sont historisées dans un journal d’audit. Les actions de création, mise à jour, suppression ou changement d’état peuvent ainsi être tracées avec un acteur, un type d’entité, un identifiant métier et, selon les cas, un état avant/après.

En parallèle, un module de reporting agrège les données commerciales et support pour produire des indicateurs de pilotage : répartition des leads par source, synthèse du pipeline, valeur gagnée, nombre de tickets ouverts, tickets en retard et temps moyen de résolution. Cette lo-

gique a été pensée comme un service transverse de synthèse plutôt que comme une simple vue d'interface.

4.3.5 Notifications email

Le backend intègre également un mécanisme de notifications SMTP. Celui-ci est utilisé pour prévenir le client lors de certains événements clés, notamment la création d'un devis ou l'ajout d'un message public sur un ticket. Ce choix renforce la continuité de service et évite au client de dépendre exclusivement d'une consultation active du portail.

4.4 Réalisation du Frontend (React)

Le frontend web a été développé sous forme d'application SPA avec React et Vite. L'interface a été structurée autour de deux espaces distincts : un espace interne destiné aux utilisateurs métiers et un portail dédié aux clients. Cette séparation s'exprime à la fois dans la navigation, dans la présentation et dans la logique d'accès.

4.4.1 Pages principales

L'application interne regroupe les pages nécessaires au pilotage commercial, au support et à l'administration. Le portail client web fournit quant à lui un ensemble plus restreint, centré sur la consultation, les interactions de support et l'accès au profil.

TABLEAU 4.2 – Organisation fonctionnelle du frontend web

Espace	Pages et fonctionnalités principales
CRM interne	Tableau de bord, comptes, contacts, leads, deals, devis, tickets, activités, audit, utilisateurs, profil
Portail client web	Tableau de bord client, tickets, devis, profil, assistant conversationnel intégré

Capture d'écran — Page de connexion

Espace réservé à l'insertion de la capture de l'écran de connexion avec email, mot de passe et profils de démonstration.

FIGURE 4.1 – Capture d'écran — Page de connexion

Capture d'écran — Tableau de bord interne

Espace réservé à l'insertion de la capture du tableau de bord affichant les KPI commerciaux et support.

FIGURE 4.2 – Capture d'écran — Tableau de bord interne

Capture d'écran — Module Leads / Deals

Espace réservé à l'insertion d'une capture mettant en avant la gestion des leads, la conversion et le pipeline des deals.

FIGURE 4.3 – Capture d'écran — Module Leads / Deals

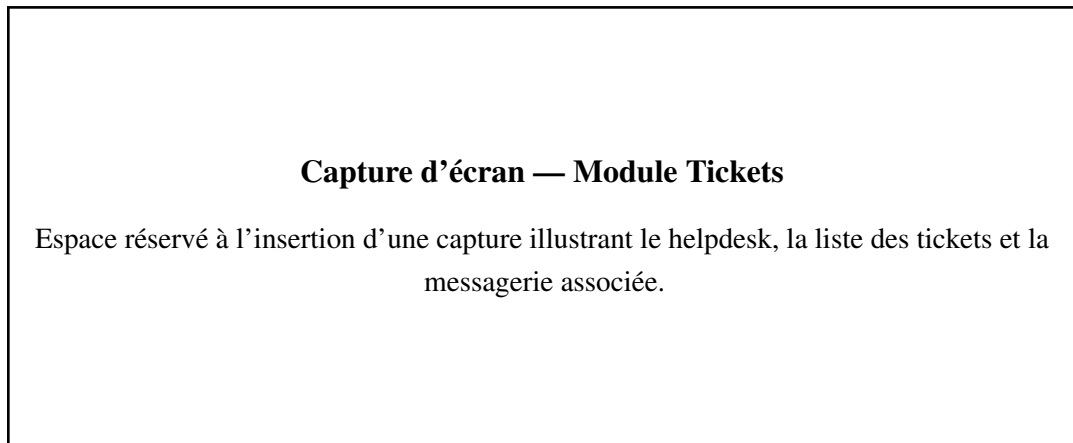


FIGURE 4.4 – Capture d'écran — Module Tickets

4.4.2 Portail Client

Le portail client bénéficie d'un layout spécifique distinct du CRM interne. Il reprend l'identité générale de l'application tout en simplifiant les parcours autour des besoins essentiels : consultation des tickets, accès aux devis, consultation du profil et assistance documentaire.

Cette séparation présente plusieurs avantages. Elle réduit la charge cognitive pour le client, renforce la sécurité en limitant les routes disponibles et rend plus explicite la frontière entre les usages internes et les usages externes. Le portail intègre également un widget conversationnel permettant au client d'interroger la base documentaire sans quitter son espace.

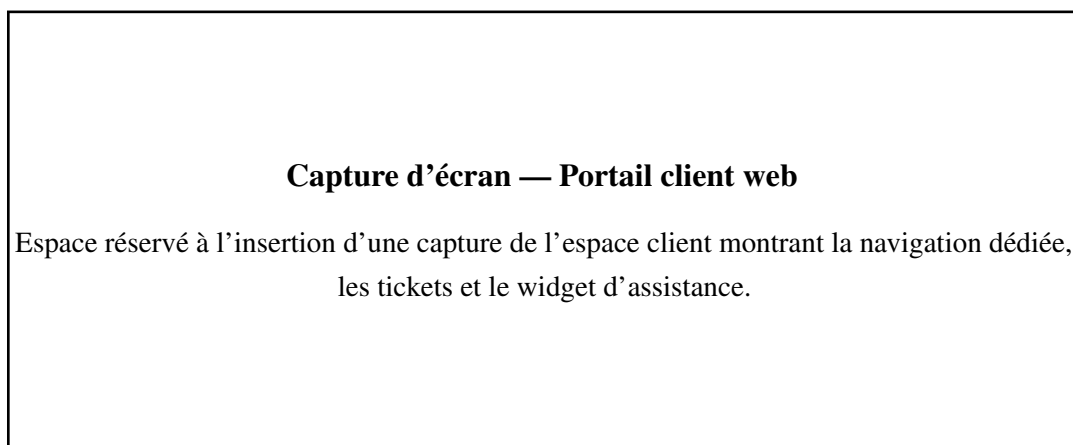


FIGURE 4.5 – Capture d'écran — Portail client web

4.4.3 Fonctionnalités UX avancées

Le frontend ne se limite pas à la restitution des données métier. Plusieurs éléments ont été mis en place pour améliorer l'expérience utilisateur :

- gestion d'un mode clair / sombre ;
- prise en charge de l'internationalisation FR/EN ;

- recherche rapide dans la navigation interne ;
- chargement différé des pages principales ;
- composant de chatbot avec conversation persistante à l'écran ;
- support de la saisie vocale dans le widget d'assistance web lorsque le navigateur le permet.

4.5 Réalisation de l'Application Mobile (React Native)

L'application mobile a été conçue comme une extension du portail client. Elle reprend les principaux parcours utiles en mobilité tout en restant volontairement plus compacte que le CRM web.

4.5.1 Technologies utilisées

- **React Native + Expo** pour le développement multiplateforme ;
- **React Navigation** avec navigation *Native Stack* ;
- **Axios** pour les appels API ;
- **Expo Secure Store** pour le stockage sécurisé du jeton ;
- **Context API** pour l'authentification et le thème.

4.5.2 Écrans de l'application

L'application mobile expose six écrans fonctionnels principaux, auxquels s'ajoute un écran de blocage pour les rôles internes non autorisés à utiliser cette interface.

TABLEAU 4.3 – Écrans principaux de l'application mobile

Écran	Description
LoginScreen	Connexion de l'utilisateur client avec récupération du profil
ClientDashboardScreen	Vue synthétique du compte, des tickets et des devis
ClientTicketsScreen	Consultation, création et suivi des tickets du compte client
ClientQuotesScreen	Consultation des devis associés au compte
ClientChatbotScreen	Interrogation de l'assistant documentaire depuis le mobile
ProfileScreen	Consultation et mise à jour des informations de profil
NotClientScreen	Message d'information pour les rôles internes invités à utiliser la version web

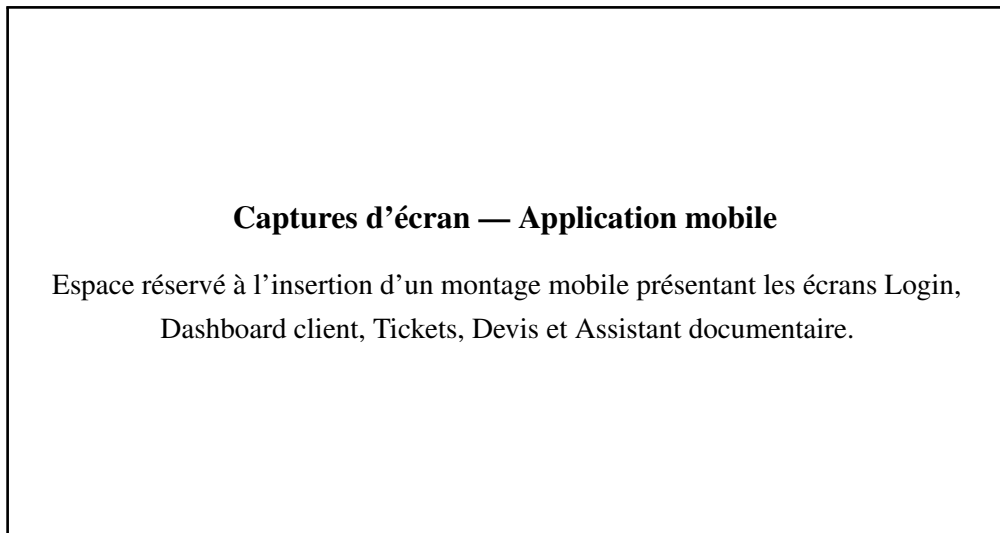


FIGURE 4.6 – Captures d'écran — Application mobile

4.6 Réalisation du Chatbot IA (RAG)

Le module d'assistance ne s'appuie pas sur des requêtes directes vers la base métier. Il repose sur une architecture documentaire orientée RAG, ce qui permet de fournir des réponses sur des contenus d'entreprise sans exposer directement la base transactionnelle.

4.6.1 Architecture RAG

Le fonctionnement du module peut être résumé en cinq étapes :

1. constitution d'une base documentaire métier en Markdown ;
2. indexation de cette base dans une collection vectorielle locale ;
3. recherche sémantique des passages les plus pertinents pour une question ;
4. génération d'une réponse contextualisée lorsque le fournisseur LLM est disponible ;
5. repli sur une réponse documentaire synthétique sinon.

4.6.2 Implémentation

L'implémentation repose sur plusieurs composants complémentaires :

TABEAU 4.4 – Composants techniques du module RAG

Composant	Responsabilité
chatbot_router.py	Exposer les endpoints d'interrogation et d'état du module
rag_chain.py	Orchestrer la recherche documentaire et la construction de la réponse
vector_store.py	Gérer la collection ChromaDB et la recherche par similarité
llm_client.py	Dialoguer avec le fournisseur LLM lorsqu'il est configuré
rag_documents/	Héberger les documents sources de la base de connaissances

4.6.3 Documents RAG

La base documentaire du chatbot est composée de documents métiers ciblés, destinés à couvrir les informations les plus fréquemment utiles pour les utilisateurs.

TABLEAU 4.5 – Documents utilisés par la base de connaissances documentaire

Document	Rôle
01_presentation_frs.md	Présentation générale de l'entreprise et de son positionnement
02_services_et_offres.md	Catalogue synthétique des services et offres proposés
03_faq_clients.md	Réponses aux questions fréquentes côté client
04_conditions_generales.md	Informations de cadrage contractuel et conditions générales
05_guide_support.md	Procédures et bonnes pratiques liées au support
06_references_projets.md	Exemples de réalisations et références de projets

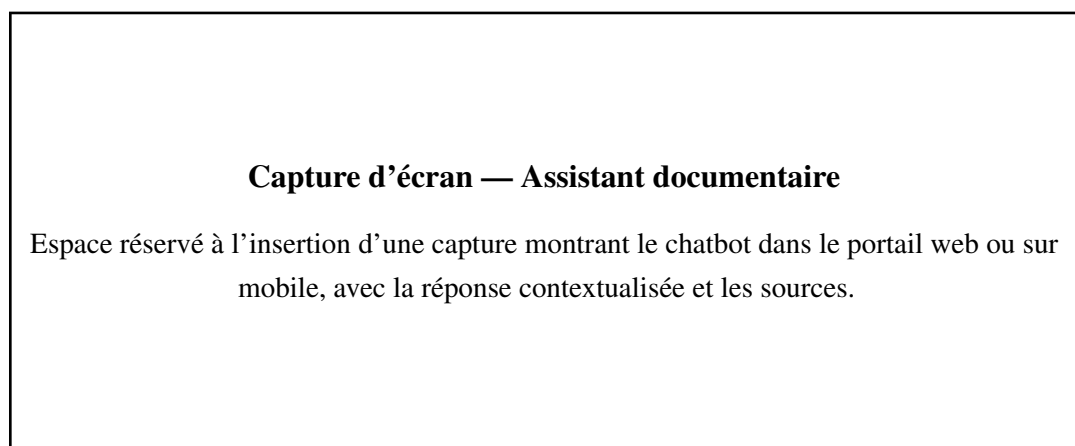


FIGURE 4.7 – Capture d'écran — Assistant documentaire

4.7 Déploiement Docker

L'exécution locale du système repose sur Docker Compose. La configuration de déploiement assemble trois services principaux : la base de données MySQL, le backend FastAPI et le frontend servi par Nginx. Des profils supplémentaires sont prévus pour lancer les campagnes de tests backend, frontend et mobile.

Ce choix présente un double intérêt. D'une part, il facilite la démonstration du projet dans un environnement reproductible. D'autre part, il prépare une industrialisation ultérieure en séparant clairement les responsabilités des conteneurs et les variables d'environnement utilisées.

TABLEAU 4.6 – Services de l’infrastructure Docker Compose

Service	Port	Rôle
db	3307 :3306	Base de données MySQL du CRM
backend	8000 :8000	API REST, logique métier et initialisation des composants RAG
frontend	3000 :80	Interface React construite et servie par Nginx
backend_unit_tests	—	Exécution des tests unitaires backend
backend_integration_tests	—	Exécution des tests d’intégration
frontend_tests	—	Exécution des tests frontend
mobile_tests	—	Exécution des tests mobile

4.8 Résultats du Sprint

TABLEAU 4.7 – Livrables du Sprint 03

Livable	Statut
Backend métier structuré et API REST opérationnelle	✓
Frontend web interne et portail client développés	✓
Application mobile client réalisée	✓
Module d’assistance documentaire intégré	✓
Infrastructure locale conteneurisée préparée	✓

4.9 Conclusion

Le troisième sprint a concrétisé les choix de conception du chapitre précédent. Le backend, le frontend, le mobile, le portail client, les notifications et le module d’assistance sont désormais réalisés et intégrés dans une même architecture. Le chapitre suivant porte sur la validation de ces réalisations, la revue des permissions, les tests et la phase de stabilisation du projet.

Sprint 04

Tests, Validation et Déploiement

Chapitre

5

Sprint 04 : Tests, Validation et Déploiement

Sommaire

5.1	Introduction	51
5.2	Objectifs du Sprint	51
5.3	Tests fonctionnels	51
5.3.1	Dispositif de tests automatisés	51
5.3.2	Matrice de tests RBAC	52
5.3.3	Scénario de test end-to-end	52
5.4	Bugs identifiés et corrigés	52
5.4.1	Méthodologie de correction	53
5.5	Validation du chatbot IA	53
5.6	Résultats du Sprint	54
5.7	Conclusion	54

5.1 Introduction

Ce dernier sprint est consacré à la consolidation du système. Après la phase de réalisation, il devient nécessaire de vérifier la cohérence fonctionnelle des modules, de confirmer la séparation des droits selon les rôles, de corriger les anomalies de stabilisation et de préparer une exécution locale reproductible de l'ensemble de la solution.

5.2 Objectifs du Sprint

- vérifier les parcours métier essentiels sur les différents modules ;
- contrôler la cohérence des permissions RBAC ;
- stabiliser les interactions entre backend, frontend, portail client et mobile ;
- valider le comportement du module d'assistance documentaire ;
- préparer une exécution locale reproductible via Docker Compose.

5.3 Tests fonctionnels

5.3.1 Dispositif de tests automatisés

Le projet intègre plusieurs niveaux de test afin de couvrir les couches principales du système :

TABLEAU 5.1 – Stratégie de tests du projet

Niveau	Outil	Couverture visée
Backend unitaire	Pytest	Authentification, logique métier et scénarios principaux du CRM
Backend intégration	Pytest + HTTP	Vérification de l'API, santé de l'application et parcours d'intégration
Frontend	Vitest	Comportements de composants, services et contextes web
Mobile	Jest	Vérification des services et comportements clés côté application mobile
Orchestration	Docker Compose + script PowerShell	Lancement coordonné de la pile d'exécution et des campagnes de tests

En complément des tests automatisés, des scénarios fonctionnels ont été revus manuellement pour valider les flux multi-rôles les plus sensibles.

5.3.2 Matrice de tests RBAC

La matrice suivante synthétise les autorisations principales observées dans l'implémentation actuelle du système.

TABLEAU 5.2 – Synthèse des permissions RBAC

Opération	Admin	Manager	Comm.	Support	Client
Connexion et accès au profil	✓	✓	✓	✓	✓
Accès au CRM interne	✓	✓	✓	✓	×
Création / mise à jour des leads	✓	✓	✓	✓	×
Conversion d'un lead qualifié	✓	✓	✓	×	×
Gestion des comptes, contacts et activités	✓	✓	✓	✓	×
Création d'un ticket interne	✓	✓	×	✓	×
Création d'un ticket via portail client	×	×	×	×	✓
Consultation des rapports ventes	✓	✓	✓	×	×
Consultation des rapports support	✓	✓	×	✓	×
Consultation du journal d'audit	✓	✓	×	×	×
Gestion des utilisateurs	✓	×	×	×	×
Utilisation du chatbot documentaire	✓	✓	✓	✓	✓

5.3.3 Scénario de test end-to-end

Pour vérifier la cohérence globale du système, un scénario transverse a été retenu :

1. un administrateur crée les utilisateurs internes nécessaires et rattache un utilisateur client à un compte ;
2. un commercial crée un lead, le qualifie puis le convertit en compte, contact et deal ;
3. un commercial crée un devis lié au deal ;
4. le client consulte le devis depuis son portail ;
5. le client ouvre un ticket de support ;
6. un agent support ajoute une note interne puis une réponse publique ;
7. le client visualise uniquement les réponses publiques et peut ajouter un nouveau message ;
8. un manager consulte les rapports de vente et de support ;
9. un administrateur vérifie la trace des opérations dans l'audit ;
10. le client utilise l'assistant documentaire pour poser une question relative au support ou aux services.

5.4 Bugs identifiés et corrigés

La phase de stabilisation a permis d'identifier plusieurs points d'amélioration concrets, touchant à la sécurité, à l'ergonomie et à la cohérence entre interface et API.

TABLEAU 5.3 – Exemples d’anomalies de stabilisation traitées

ID	Zone	Symptôme observé	Correction apportée	Statut
BUG-1	Utilisateurs	Un utilisateur authentifié pouvait consulter un autre profil sans contrôle suffisant	Restriction de l'accès au profil de l'utilisateur lui-même ou aux rôles de supervision autorisés	✓
BUG-2	Tickets / Frontend	Les agents support ne disposaient pas d'une liste exploitable des utilisateurs assignables	Ajout d'un endpoint dédié aux utilisateurs assignables et raccordement du formulaire de ticket	✓
BUG-3	Tickets / UI	Le formulaire de création proposait des champs non cohérents avec l'API de création	Alignement du comportement du formulaire avec la logique backend selon création ou modification	✓
BUG-4	Tickets / Messagerie	Le nom de l'auteur n'était pas correctement restitué dans l'affichage des messages	Enrichissement de la réponse API pour inclure l'auteur lisible côté interface	✓
BUG-5	Reporting support	Le calcul du temps moyen de résolution dépendait d'une fonction SQL spécifique au moteur MySQL	Remplacement par un calcul portable au niveau applicatif afin d'améliorer la robustesse des tests et des exécutions	✓

5.4.1 Méthodologie de correction

Pour chaque anomalie identifiée, une même démarche a été appliquée :

1. reproduction du problème sur l'interface ou via l'API ;
2. isolement de la cause dans la couche concernée ;
3. correction locale en privilégiant la cohérence avec l'architecture existante ;
4. vérification fonctionnelle du comportement corrigé ;
5. contrôle rapide des modules adjacents afin de limiter les régressions.

5.5 Validation du chatbot IA

Le module d'assistance a été évalué sur des questions cohérentes avec la nature documentaire de sa base de connaissances.

TABLEAU 5.4 – Validation fonctionnelle du module d’assistance documentaire

Question posée	Type de réponse attendue	Validation
Quels services propose FRS ?	Présentation synthétique des offres de l’entreprise	✓
Comment fonctionne le support client ?	Rappel des principes du guide support et du portail	✓
Quels sont les documents ou références de projets disponibles ?	Restitution des références et projets mis en avant	✓
Quelles informations trouve-t-on dans les conditions générales ?	Réponse orientée vers le document contractuel correspondant	✓

5.6 Résultats du Sprint

TABLEAU 5.5 – Livrables du Sprint 04

Livable	Statut
Dispositif de tests automatisés backend, frontend et mobile structuré	✓
Matrice RBAC revue sur les parcours principaux	✓
Anomalies de stabilisation corrigées	✓
Validation du portail client et des flux tickets	✓
Validation du chatbot documentaire sur des cas d’usage réalistes	✓
Configuration de déploiement local Docker Compose finalisée	✓

5.7 Conclusion

Le quatrième sprint a permis de consolider le projet et de le rendre plus robuste. La revue fonctionnelle a confirmé la cohérence des grands flux, la vérification RBAC a clarifié le périmètre de chaque rôle, et la phase de stabilisation a amélioré la sécurité ainsi que la cohérence entre interfaces et API. Le projet dispose désormais d’une base documentaire, technique et fonctionnelle nettement plus solide pour une démonstration ou une poursuite de développement.

Conclusion Générale



Conclusion Générale

Ce stage de fin d'études m'a permis de conduire un projet logiciel complet, depuis l'analyse du besoin jusqu'à la structuration d'une solution multi-interface prête à être démontrée et consolidée. Le système développé pour **FRS** répond à un besoin concret de centralisation de la relation client et propose une vision unifiée des activités commerciales, du support et des interactions avec les clients.

Bilan du travail réalisé. Le projet a été mené de manière progressive à travers quatre sprints complémentaires :

- le premier sprint a permis d'étudier l'existant, d'identifier les acteurs et de formaliser les besoins ;
- le deuxième sprint a servi à concevoir les modèles de données, les diagrammes UML, l'architecture technique et l'API REST ;
- le troisième sprint a été consacré à l'implémentation du backend, du frontend, de l'application mobile, du portail client et du module d'assistance documentaire ;
- le quatrième sprint a porté sur la validation fonctionnelle, la revue des permissions, la correction des anomalies de stabilisation et la préparation du déploiement local via Docker Compose.

Résultats obtenus. La solution développée fournit aujourd'hui :

- un cycle commercial structuré autour des leads, comptes, contacts, deals et devis ;
- un module de support avec tickets, statuts, priorités et messagerie publique ou interne ;
- un portail client web sécurisé pour le suivi des tickets et des devis ;
- une application mobile client permettant l'accès nomade aux fonctionnalités essentielles ;
- des tableaux de bord de pilotage pour les ventes et le support ;
- un journal d'audit pour assurer la traçabilité des opérations sensibles ;
- une base documentaire interrogeable via un assistant conversationnel fondé sur la technique RAG ;
- une infrastructure locale conteneurisée facilitant l'exécution du projet et de ses campagnes de tests.

Apports techniques et professionnels. Ce projet m'a permis de consolider plusieurs compétences complémentaires : conception d'API avec FastAPI, modélisation de données relationnelles, développement d'interfaces web avec React, développement mobile avec React Native,

gestion de l'authentification JWT, séparation des responsabilités par rôles, intégration d'un moteur de recherche documentaire assisté par IA, et organisation d'un projet logiciel selon une logique itérative. Il m'a également appris à relier des besoins métier concrets à des choix d'architecture réalistes et maintenables.

Perspectives. Plusieurs axes d'amélioration peuvent prolonger ce travail :

- enrichir le portail client par des notifications temps réel ;
- renforcer la stratégie de migration et d'administration de la base de données ;
- étendre la couverture de tests automatisés sur l'ensemble du cycle d'exécution ;
- connecter l'assistant à des sources de données métiers plus dynamiques selon des règles de sécurité strictes ;
- préparer un déploiement cloud industrialisé avec supervision et gestion des secrets.

En définitive, ce stage a constitué une expérience formatrice et professionnalisante. Il m'a donné l'occasion de participer à la construction d'une solution cohérente, utile et techniquement riche, tout en renforçant ma capacité à concevoir des systèmes logiciels réalistes, documentés et évolutifs.



Bibliographie

1. FastAPI Documentation officielle, Sebastián Ramírez, <https://fastapi.tiangolo.com/>
2. React Documentation officielle, Meta Platforms, <https://react.dev/>
3. React Native Documentation, Meta Platforms, <https://reactnative.dev/>
4. Expo Documentation, Expo, <https://docs.expo.dev/>
5. SQLAlchemy Documentation, Mike Bayer, <https://docs.sqlalchemy.org/>
6. MySQL 8.0 Reference Manual, Oracle Corporation, <https://dev.mysql.com/doc/refman/8.0/en/>
7. Docker Documentation, Docker Inc., <https://docs.docker.com/>
8. Nginx Documentation, F5 Nginx, <https://nginx.org/en/docs/>
9. Pydantic v2 Documentation, <https://docs.pydantic.dev/>
10. Chroma Documentation, <https://docs.trychroma.com/>
11. Groq Developer Documentation, <https://console.groq.com/docs/>
12. Vite Documentation, <https://vite.dev/guide/>
13. React Router Documentation, <https://reactrouter.com/>
14. i18next Documentation, <https://www.i18next.com/>
15. Tailwind CSS Documentation, <https://tailwindcss.com/docs>
16. JSON Web Tokens Introduction, Auth0, <https://jwt.io/introduction>
17. Scrum Guide, Ken Schwaber et Jeff Sutherland, <https://scrumguides.org/>, 2020
18. Domain-Driven Design : Tackling Complexity in the Heart of Software, Eric Evans, Addison-Wesley, 2003
19. Clean Architecture : A Craftsman's Guide to Software Structure and Design, Robert C. Martin, Pearson Education, 2017